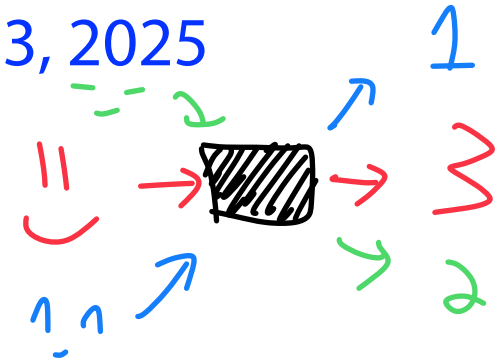
Algorithms and Data Structures for Data Science

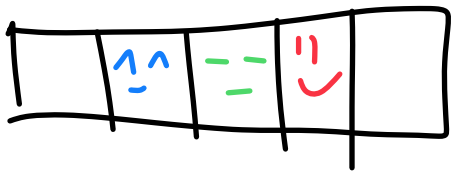
Hashing

CS 277

Brad Solomon

March 3, 2025





UNIVERSITY OF ILLINOIS URBANA-CHAMPAIGN

Department of Computer Science

Announcements

Homework 0/1 Feedback Form ends today!

MP_generate AND Informal Early Feedback form releasing today

Exam 1 next week — practice exam will be released soon*

Content can be found on website, does not include hashing

Learning Objectives

Motivate (key, value) paired datasets

Define the dictionary ADT

Motivate and define a hash table

Discuss what a 'good' hash function looks like

Identify a key weakness of the hash table

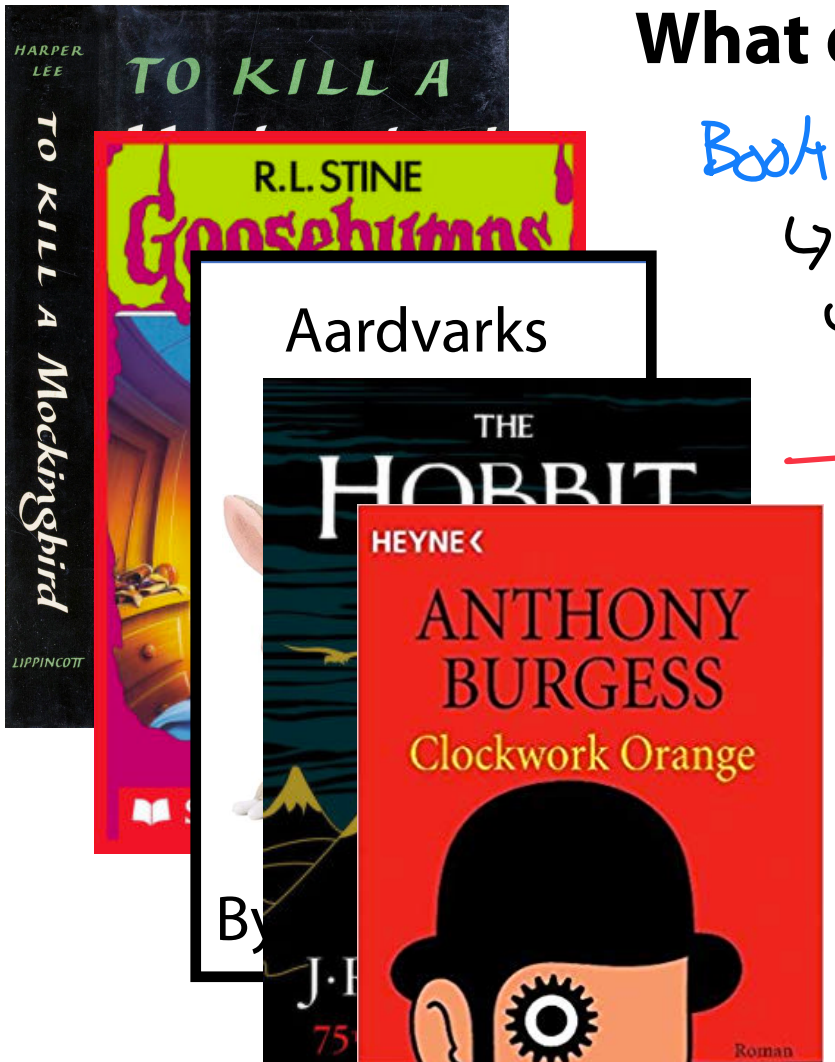
Introduce strategies to 'correct' this weakness

Python dictionaries

Implementation

Data Structure Review

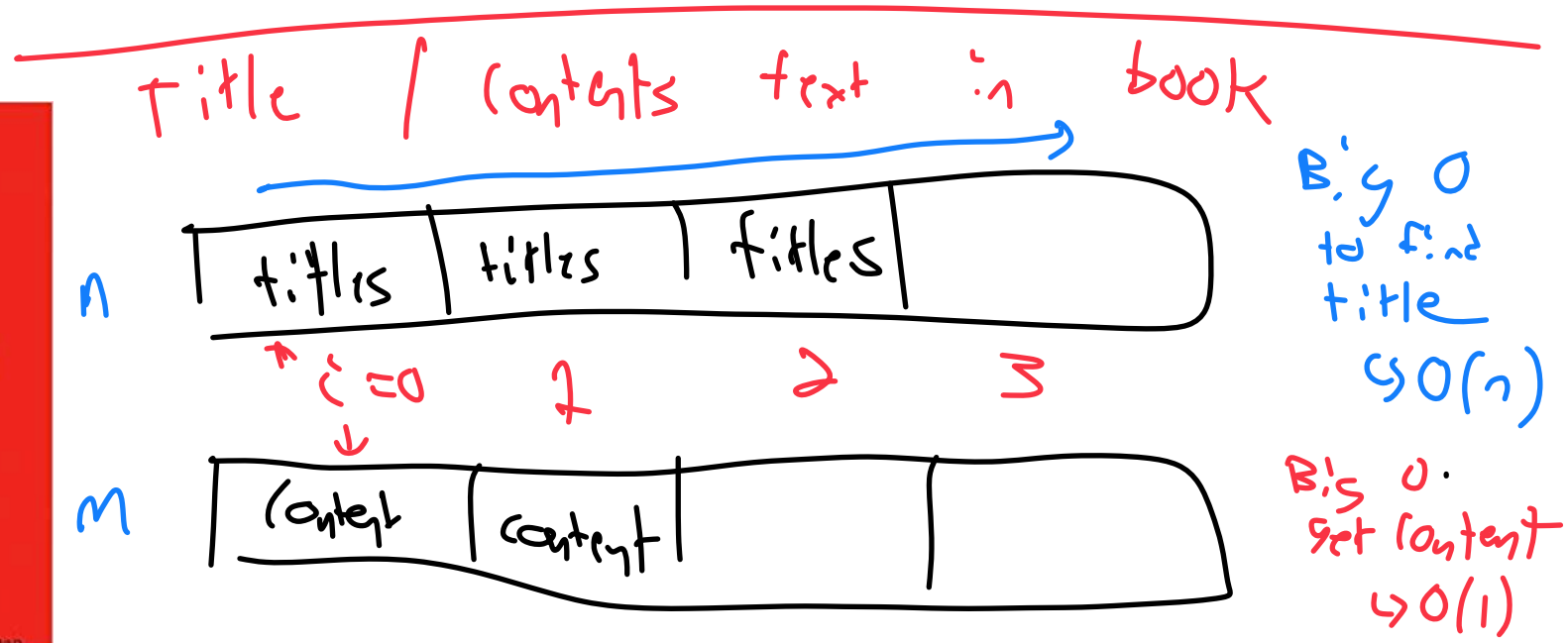
I have a collection of books and I want to store them efficiently!



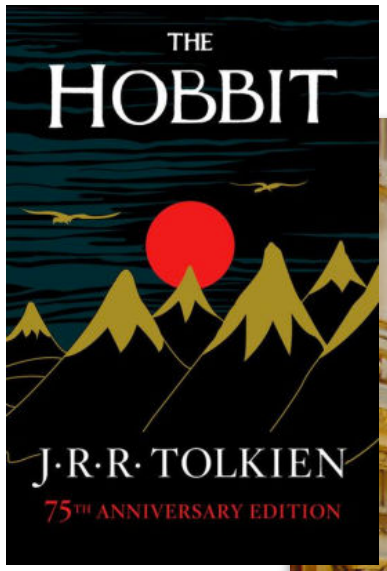
What data structures can I use here?

Book has
- A title
- An author
- Genre
- Page count

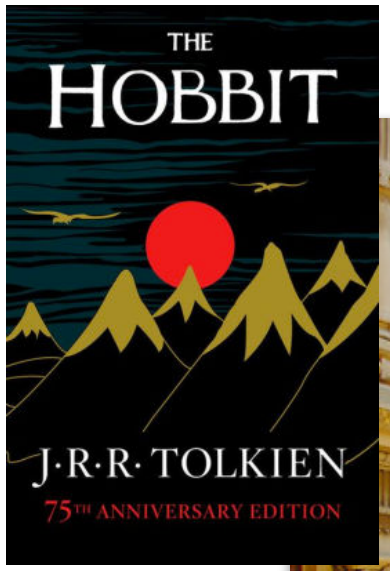
Create a book class
- Object oriented program



If we recognize that libraries are ordered: $O(log\ n)$



What if $O(log\ n)$ isn't good enough?

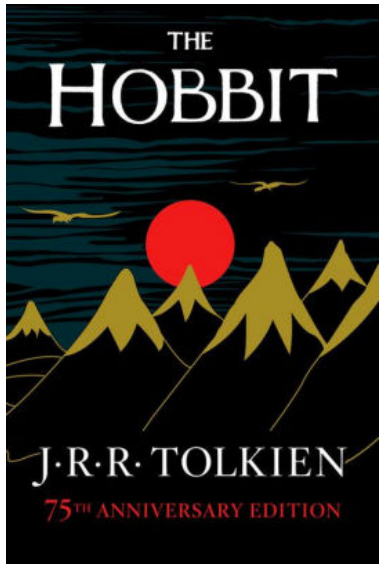


A Hash Table based Dictionary

key

(Hash Value)

Address



O(1)

O(1)

ISBN: 9781526602381

Call #: PR
6068.O93
H35 1998

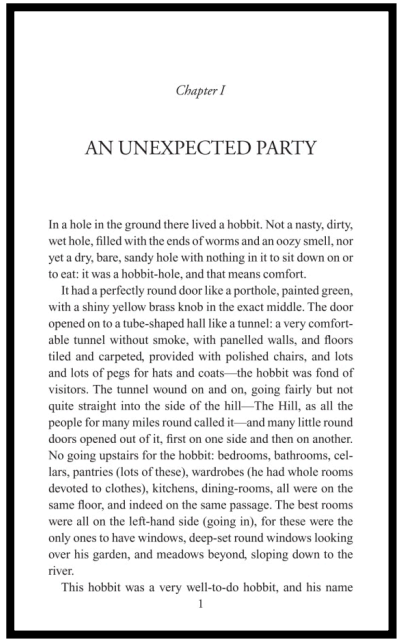
ISBN: 9781526602381

Call #: PR
6068.O93
H35 1998

O(1)

O(1)

value



Chapter I

AN UNEXPECTED PARTY

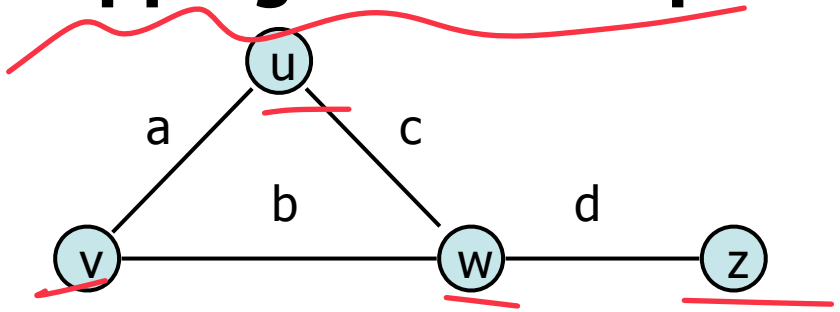
In a hole in the ground there lived a hobbit. Not a nasty, dirty, wet hole, filled with the ends of worms and an oozy smell, nor yet a dry, bare, sandy hole with nothing in it to sit down on or to eat: it was a hobbit-hole, and that means comfort.

It had a perfectly round door like a porthole, painted green, with a shiny yellow brass knob in the exact middle. The door opened on to a tube-shaped hall like a tunnel: a very comfortable tunnel without smoke, with panelled walls, and floors tiled and carpeted, provided with polished chairs, and lots and lots of pegs for hats and coats—the hobbit was fond of visitors. The tunnel wound on and on, going fairly but not quite straight into the side of the hill—The Hill, as all the people for many miles round called it—and many little round doors opened out of it, first on one side and then on another. No going upstairs for the hobbit: bedrooms, bathrooms, cellars, pantries (lots of these), wardrobes (he had whole rooms devoted to clothes), kitchens, dining-rooms, all were on the same floor, and indeed on the same passage. The best rooms were all on the left-hand side (going in), for these were the only ones to have windows, deep-set round windows looking over his garden, and meadows beyond, sloping down to the river.

This hobbit was a very well-to-do hobbit, and his name

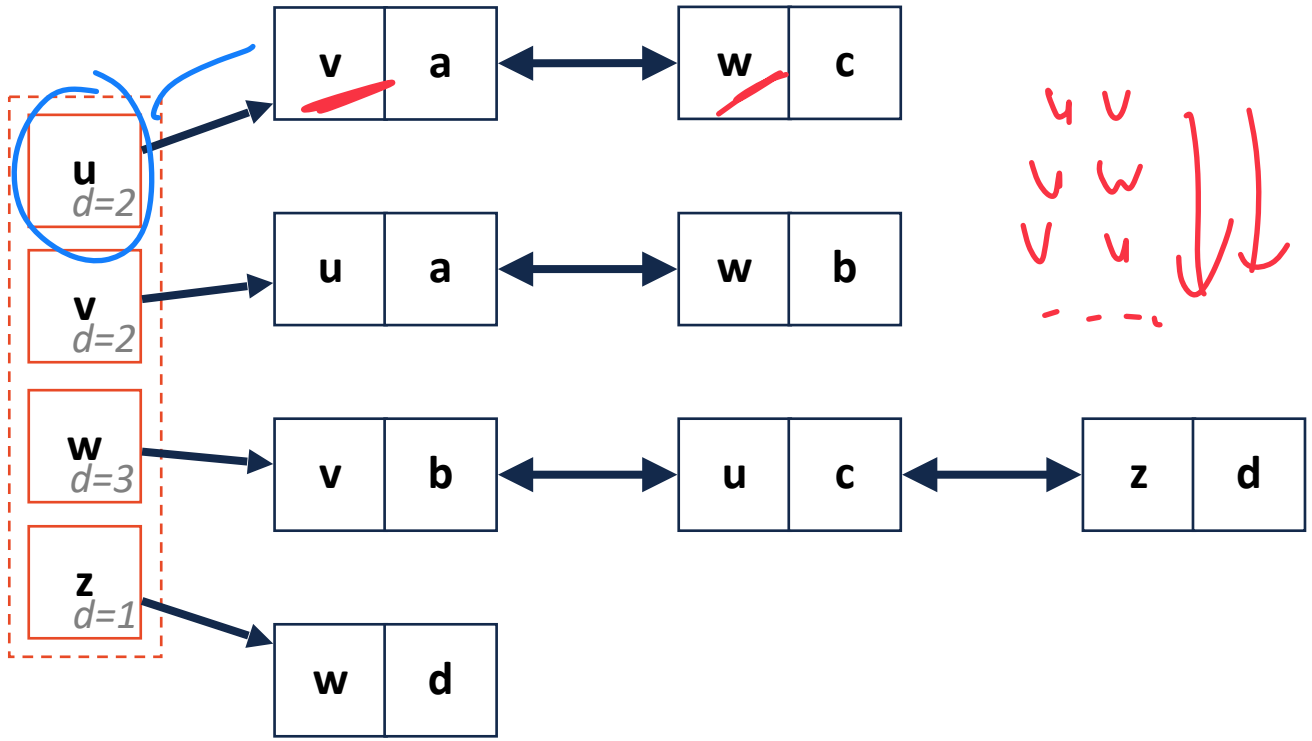
Key-Value Datasets

Mapping relationships



key to look up

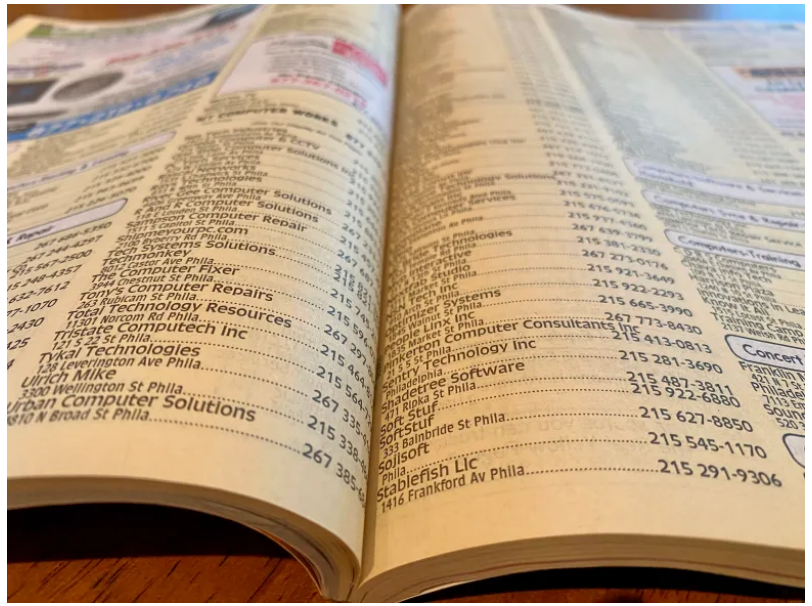
each key associated w/ value



Efficient data organization

Brad	CS 173, CS 225, CS 277
Geoffrey	CS 125, CS 233
Carl	CS 173, CS 225, CS 340
Wade	CS 107, CS 225, CS 340

Flexible relationship storage!

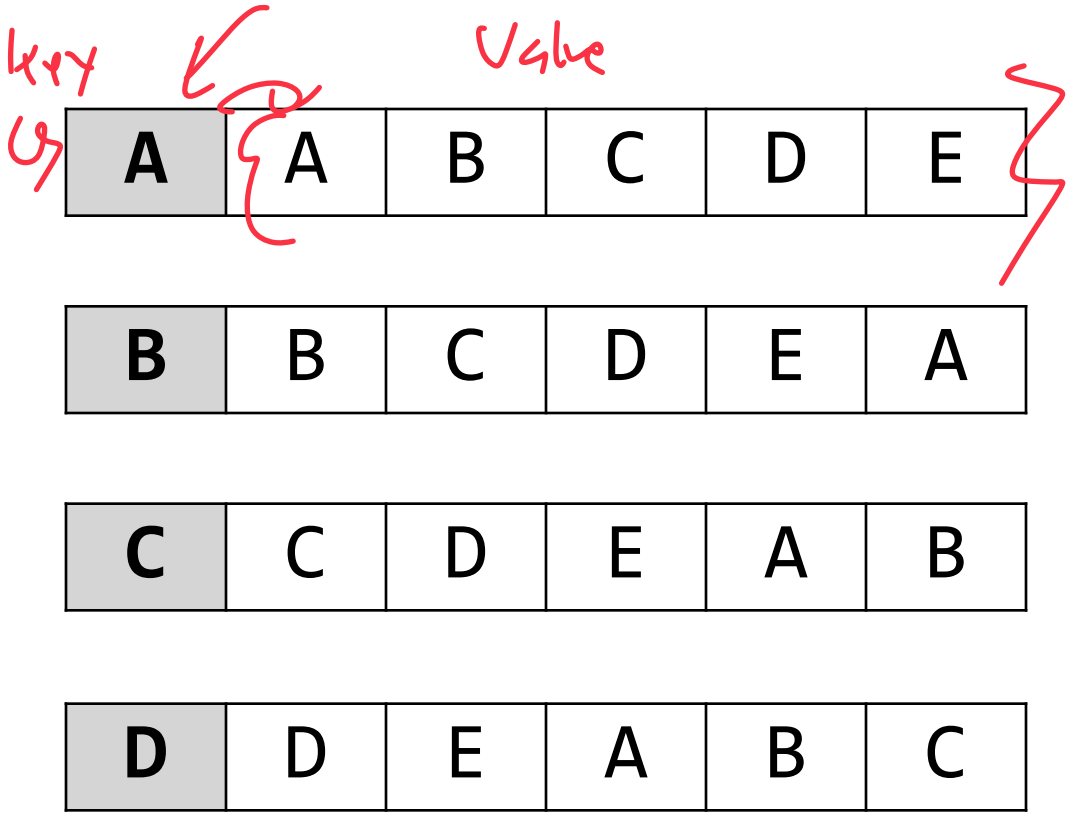


Vigenere Cipher

a **dictionary!**

While not necessary, you may find it easier to handle using a ~~**matrix!**~~

Every key is associated with a single list



Plaintext

		A	B	C	D	E
+0	**A**	A	B	C	D	E
+1	**B**	B	C	D	E	A
+2	**C**	C	D	E	A	B
…	**D**	D	E	A	B	C
Key	**E**	E	A	B	C	D

Dictionary ADT

A dictionary is an **unordered** collection of (key, value) pairs

The keys can be of any ***hashable*** (immutable) type

The values can be of any type

A minimal set of operations (that all dictionaries should have):

Dictionary ADT

A dictionary is an **unordered** collection of (key, value) pairs

The keys can be of any ***hashable*** (immutable) type

The values can be of any type

A minimal set of operations (that all dictionaries should have):

1. **Insert:** Add a new (key, value) pair to the dictionary
2. **Delete:** Remove a specific (key, value) pair from the dictionary
3. **getKeys:** Return the set of keys stored in the dictionary
4. **getValue:** Given a key, get the corresponding value
5. **Constructor:** Create a new empty dictionary

equiv to getItem

Dictionary in Python: Constructor

To create a dictionary in Python, we use **squiggly brackets '{}'**

```
emptyDictionary = {}
```

key: value

```
myDict = {"A": 1, "B": 2, "C": 3}
```

```
mixed = {"A": 1, "C": [1, 2, 3], 5: "D", 8.9: 2.1}
```

Dictionaries in Python: getData

Items in a dictionary are accessed **by key** using square brackets: [ ]

This can be used to get the corresponding value of a key:

```
1  mixed = {"A": 1, "C": [1, 2, 3], 5: "D", 1: 2.1}
2
3  print(mixed["A"])
4
5  print(mixed[1])
6
7  print(mixed["B"])
```

This can be used to update a value or to make a new (k, v) pair:

```
1  myDict = {"A": 1, "B": 2, "C": 3}
2
3  myDict["A"]="V"
4
5  myDict["D"]=3.14
```

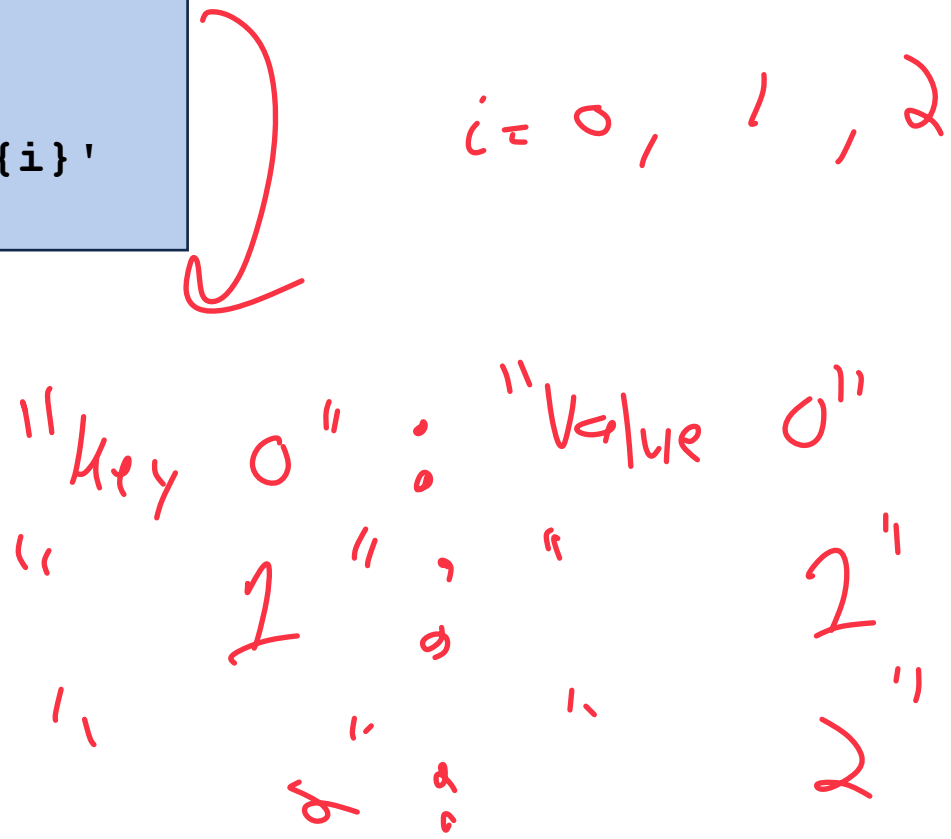
Join Code: 277

Dictionary in Python: Insert

If I want to add a new (key, value) pair, use the square bracket notation

`<dictionary>[<key>] = <value>`

```
1  empty = {}
2
3  for i in range(3):
4      empty[f'Key {i}']=f'Value {i}'
5
```



Dictionary in Python: Delete

If I want to remove an item by index, use **pop()** to remove a **key**

`<dictionary>.pop(<key>)`

```
1  myDict = {"A": 1, "B": 2, "C": 3}
2
3  myDict.pop("B")
4
5  print(myDict)
```

{'A': 1, "C": 3}

If we wanted to remove an item by value, what could we do?

Access any one value w/ its key

↳ Get all key? d.keys() ∈ list

```
for key in d.keys():
    ↳ check value of key
    if d[key] == query:
        ↳ d.pop(key)
```

Dictionaries in Python: getKeys

A dictionary stores a collection of keys in no particular order:

`<dictionary>.keys()`

```
1  myDict = {"A": 1, "B": 2, "C": 3}
2  print(myDict.keys())
3
4  print(list(myDict.keys()))
5
```

A list of keys

if you dont know if key exists

if searchKey not in d.keys():
↳ Dont search!

if search in d.keys():
d[search]

Dictionary Abstract Data Type

A minimally functional dictionary must have the following functions:

Constructor: `__init__()`

Insert: `dict[k]=v`

Delete: `pop(k)`

getValue: `[k]`

getKeys: `dict.keys()`

Caesar Cipher (as a dictionary)

Lets practice using dictionaries to reproduce an earlier lab question!

A Hash Table based Dictionary

```
1 d = {}
2 d[k] = v
3 print(d[k])
```

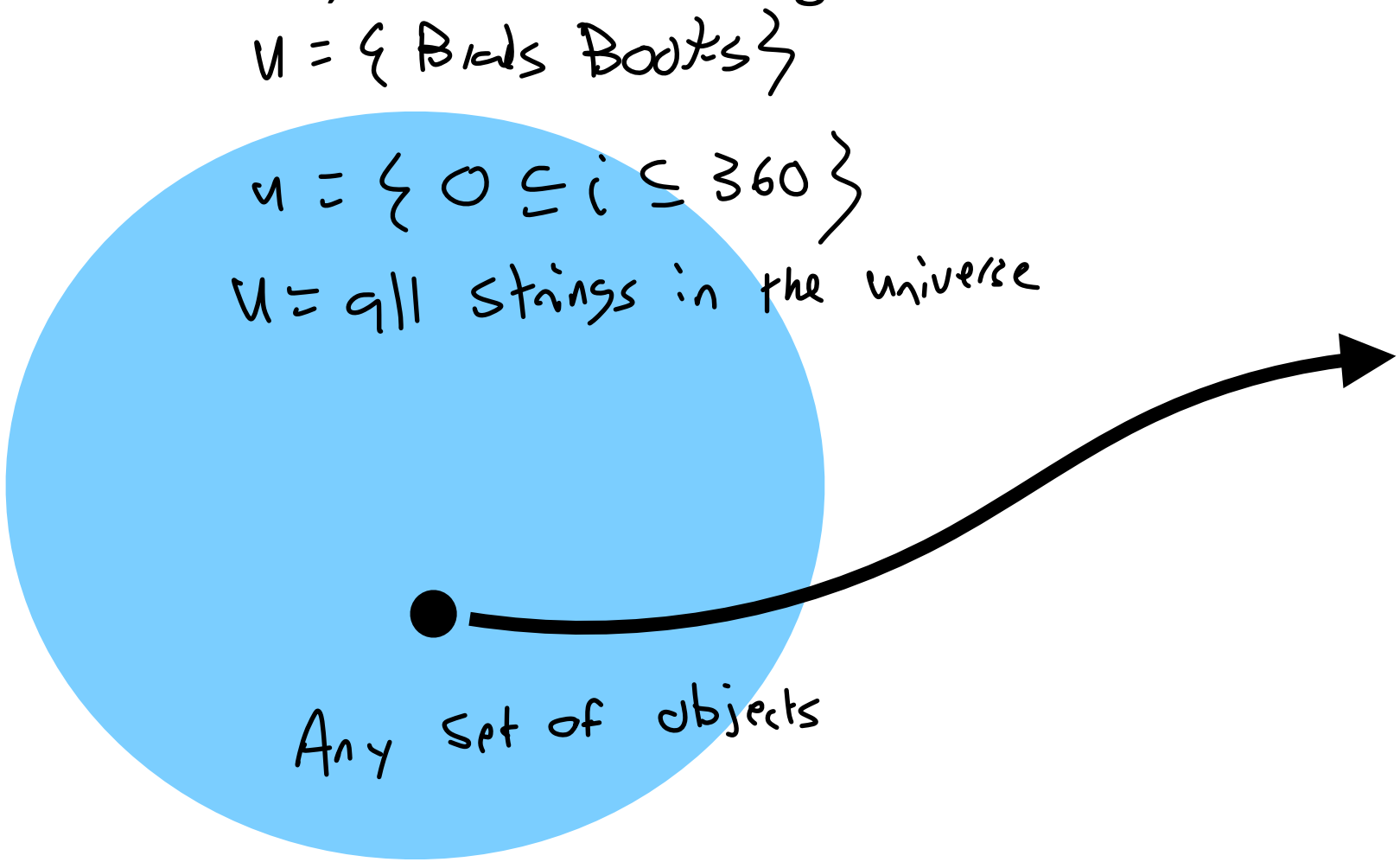
A **Hash Table** consists of three things:

1. A hash function (key → int (hash value) address)
2. A data storage structure (An array)
3. ???

Hash Function

$f("ABA") = 2$

Maps a **keyspace**, a (mathematical) description of the keys for a set of data, to a set of integers.



m elements

Key	Value

int addresses

Hash Function

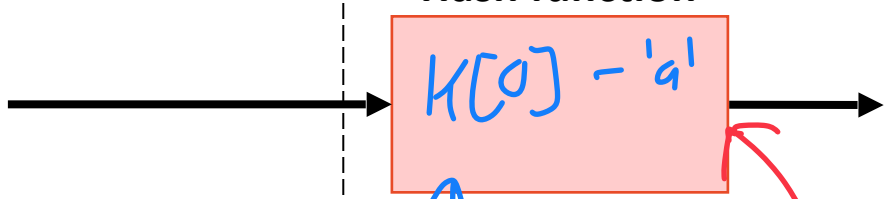
A hash function ***must*** be:

- **Deterministic**: Given the same key, output the same hash value / address
- **Efficient**: O(1)
- **Defined for a certain size table:** Create for specific sized array

Hash Function

Key value

(Gunter, CS 422)
(Angrave, CS 241)
(Beckman, CS 421)
(Challon, CS 125)
(Davis, CS 101)
(Evans, CS 225)
(Fagen-Ulmschneider, CS 107)
(Herman, CS 233)



first letter of every name

Key	Value
Angrave	241
Beckman	421
Gunter	422

a = 0
b = 1
c = 2
d ... s_{456}
f_9

h("Gunter") → 6
O(1)
O(1)

Hash Function → Stores all faculty & classes they teach

(Gunter, CS 422)
(Angrave, CS 241)
(Evans, CS 225)
(Beckman, CS 421)
(Challon, CS 125)
(Davis, CS 101)
(Fagen-Ulmschneider, CS 107)
(Herman, CS 233)

Alawini 411

Pull first letter & convert to #

Hash function

(key[0] - 'A')

A: 0
B: 1
C: :)
D: 3

an array

	Key	Value
0	Angrave	241
1	Beckman	421
	Challon	125
	Davis	101
	Evans	225
	Fagen-U	107
	Gunter	422
	Herman	233

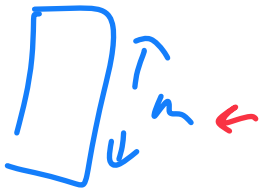
Not specific to class

Bad out of bounds

This function works for this set but not true universally

General Hash Function

An $O(1)$ deterministic operation that maps all keys in a universe U to a defined range of integers $[0, \ldots, m-1]$



- A **hash**: f(x) -> int

f(x) -> 0 - infinity
2^{64}

- A **compression**: converts big int to array sized int

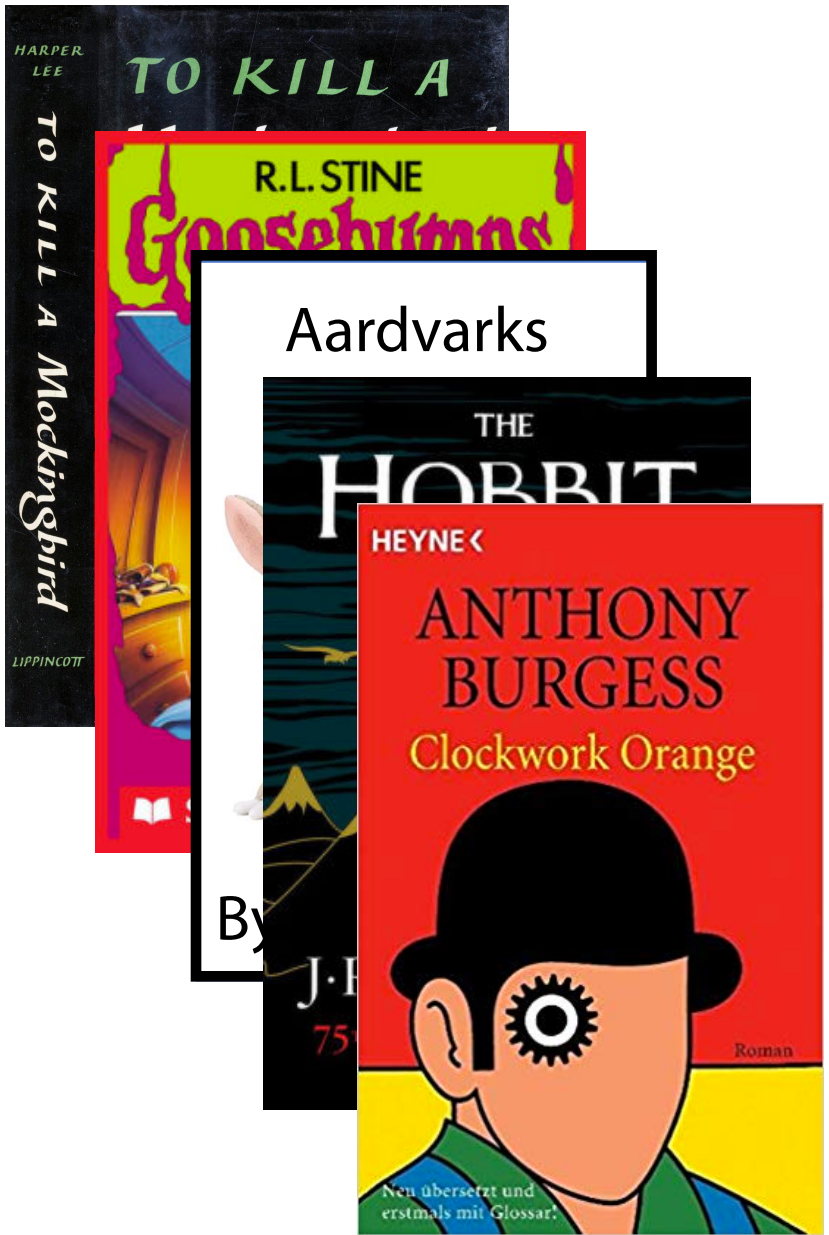
f(x) % 8
% m

Choosing a good hash function is tricky…

- Don't create your own!

Hash Function

Is this a hash (and if so how good is it)?



Get the first letter of the author's first name

Get the first letter of the author's last name

Add up the index in the alphabet for each

Is hash function!

Why not good?

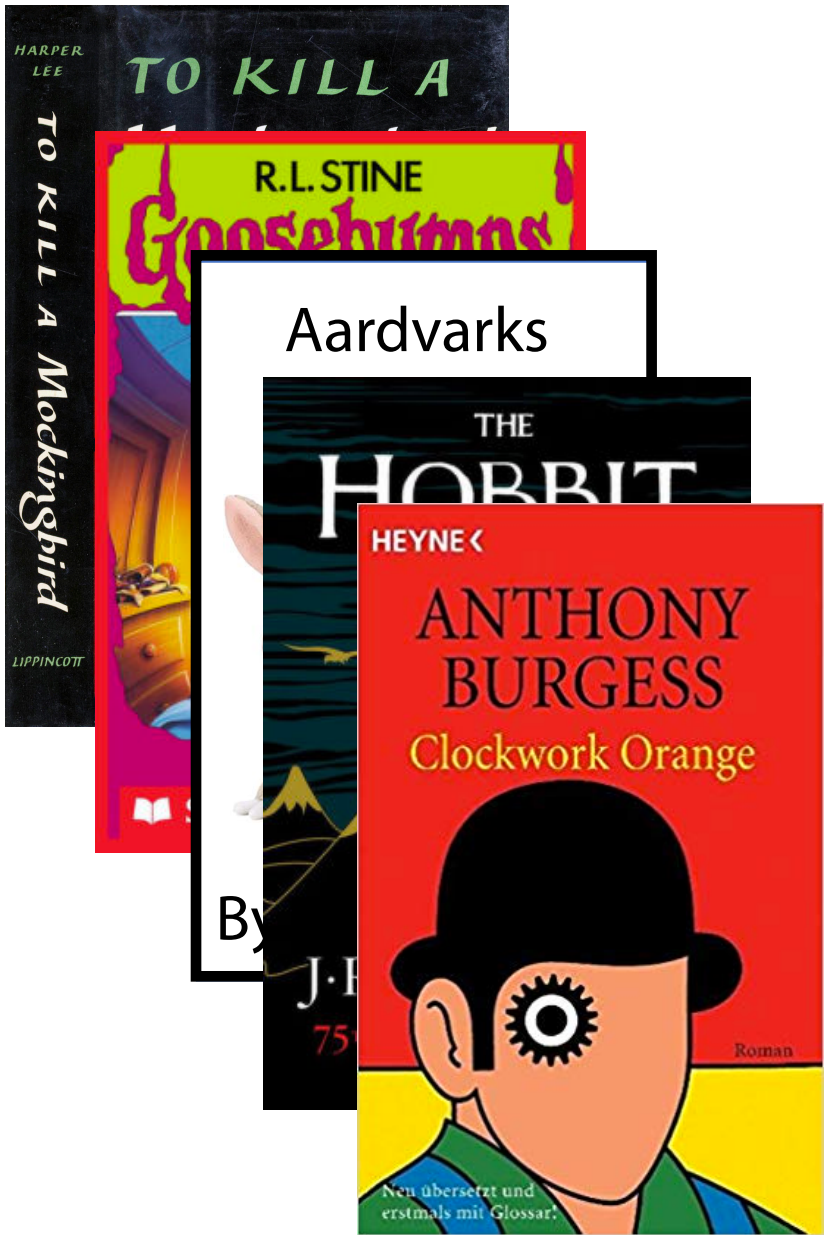
↳ Author names not truly uniform / random

↳ certain combos more likely

Join Code: 277

Hash Function

Is this a hash (and if so how good is it)?



Get the number of pages in the book

Multiply it by a randomly generated number

Not a valid hash

Book A → 100 pages * 37 → 3100

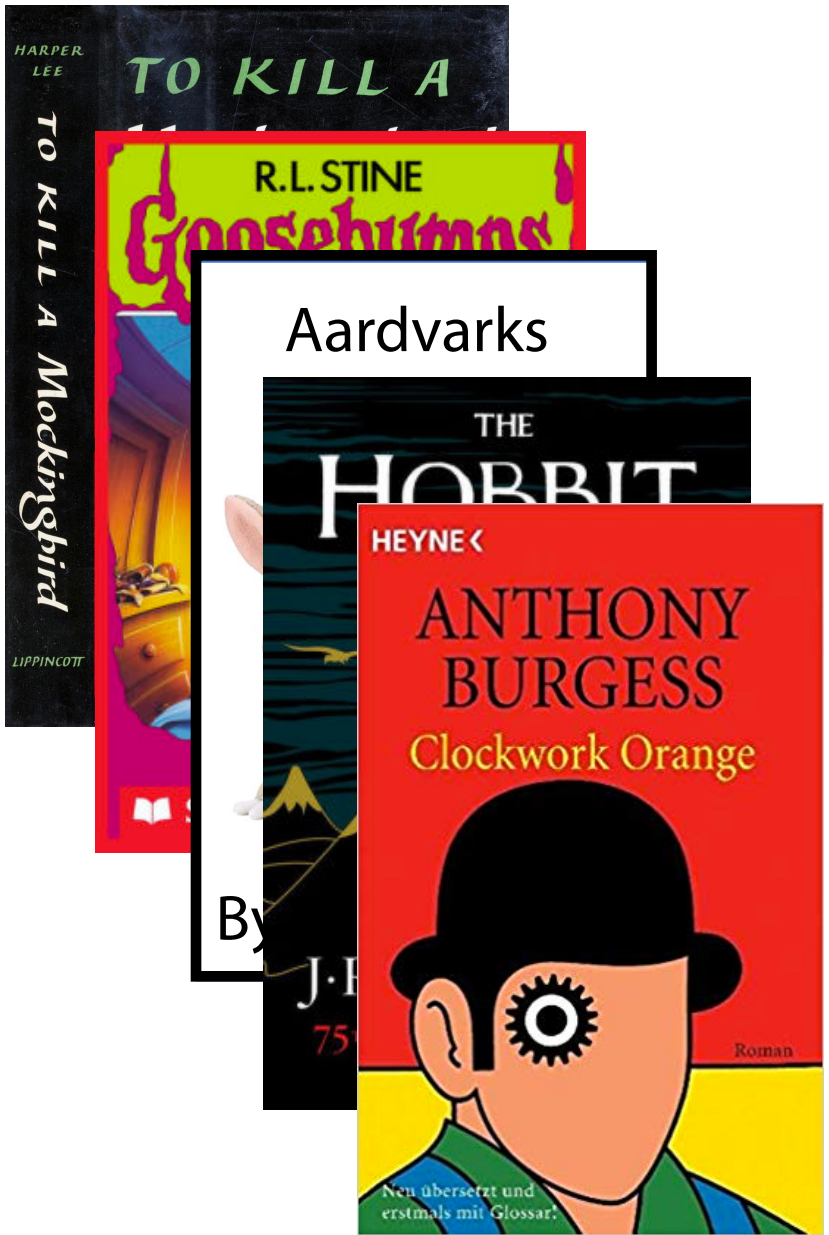
Book A → 100 * 2 → 200 ???

If I always use same random #

Join Code: 277

Hash Function

Is this a hash (and if so how good is it)?



Get every other letter of the title

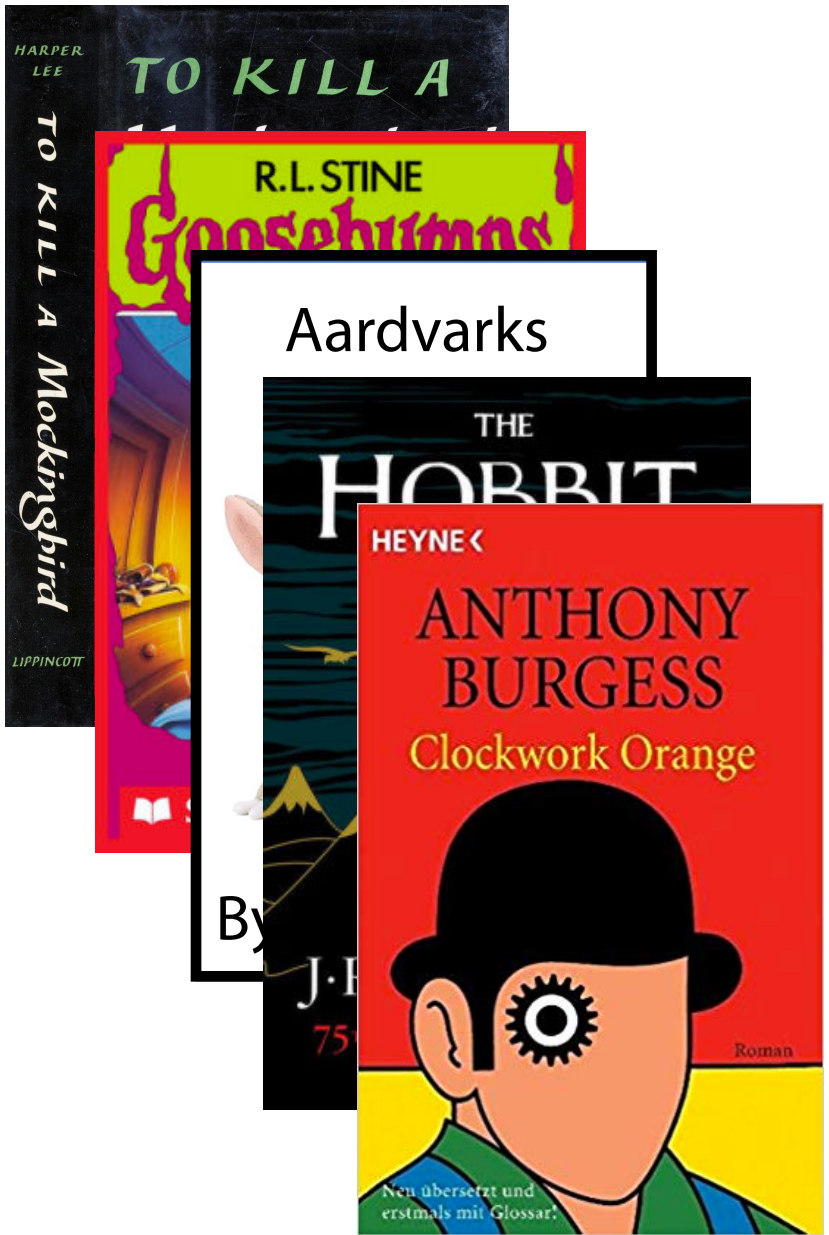
Convert all letters into numbers

A valid hash & a good one

Join Code: 277

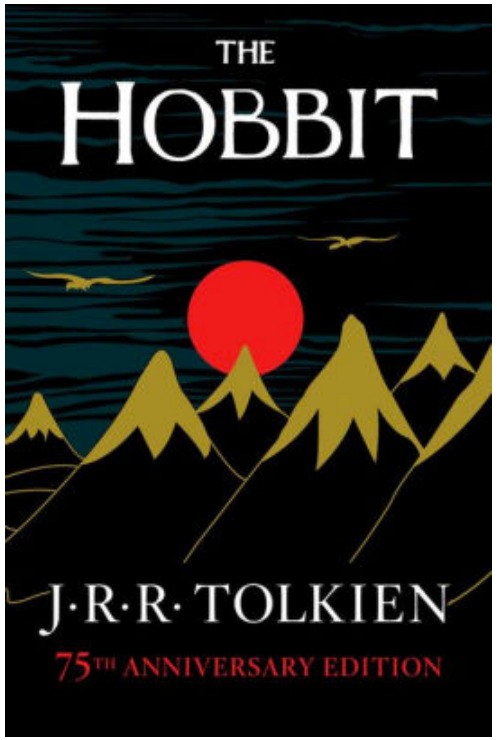
Hash Function

What are some other hash functions for books?



What defines a **good** hash function?

A Hash Table based Dictionary

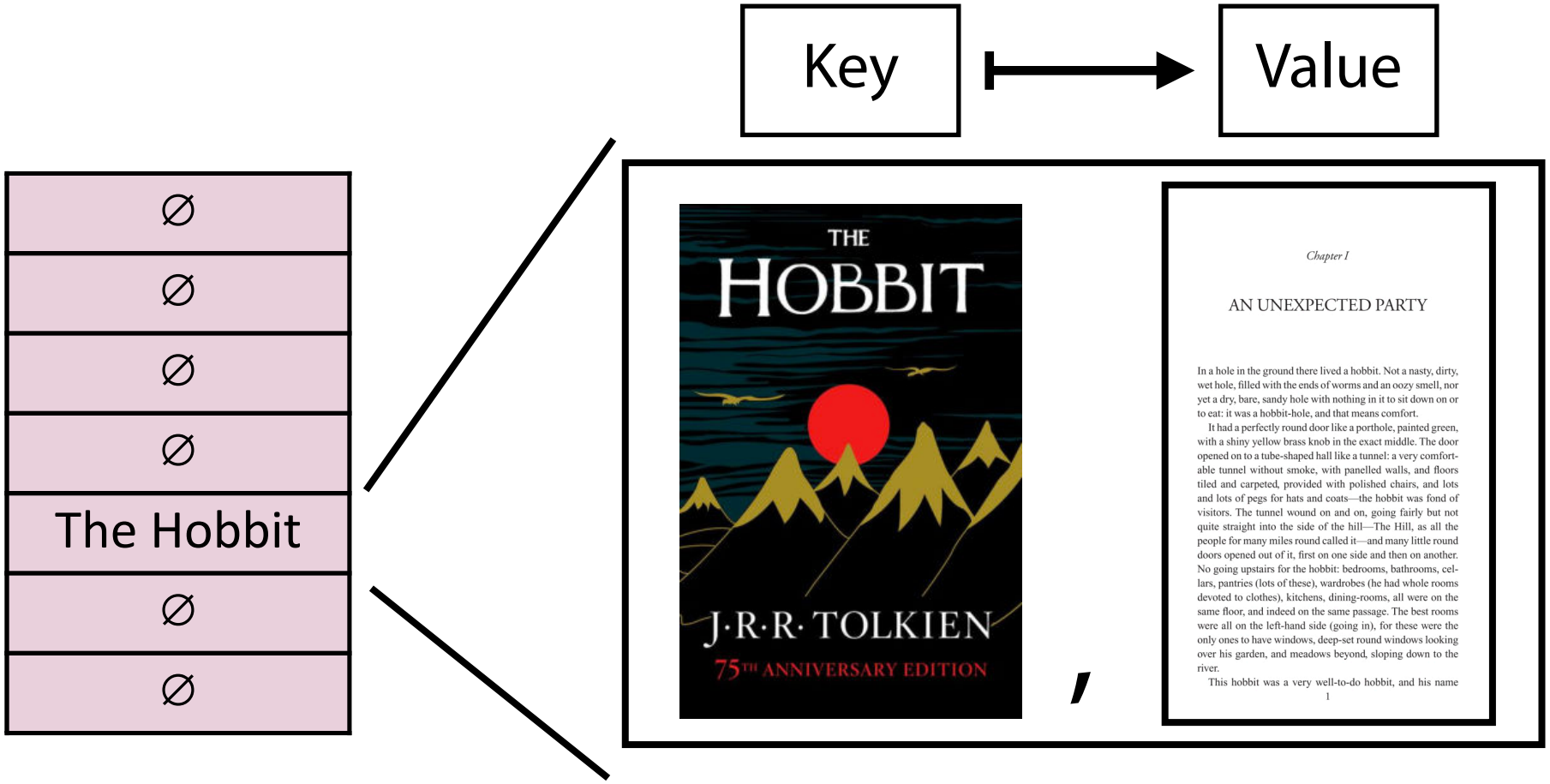


'J' + 'T' = 30

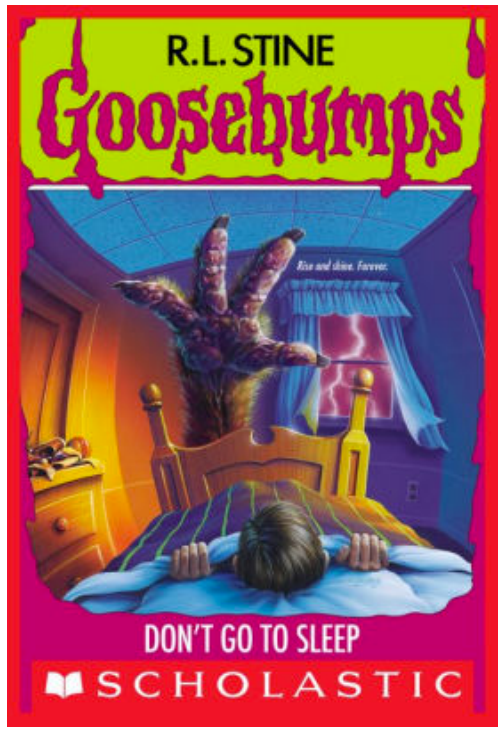
Author Name Hash Function

...	...
27	∅
28	∅
29	∅
30	The Hobbit
31	∅
...	...

A Hash Table based Dictionary



A Hash Table based Dictionary

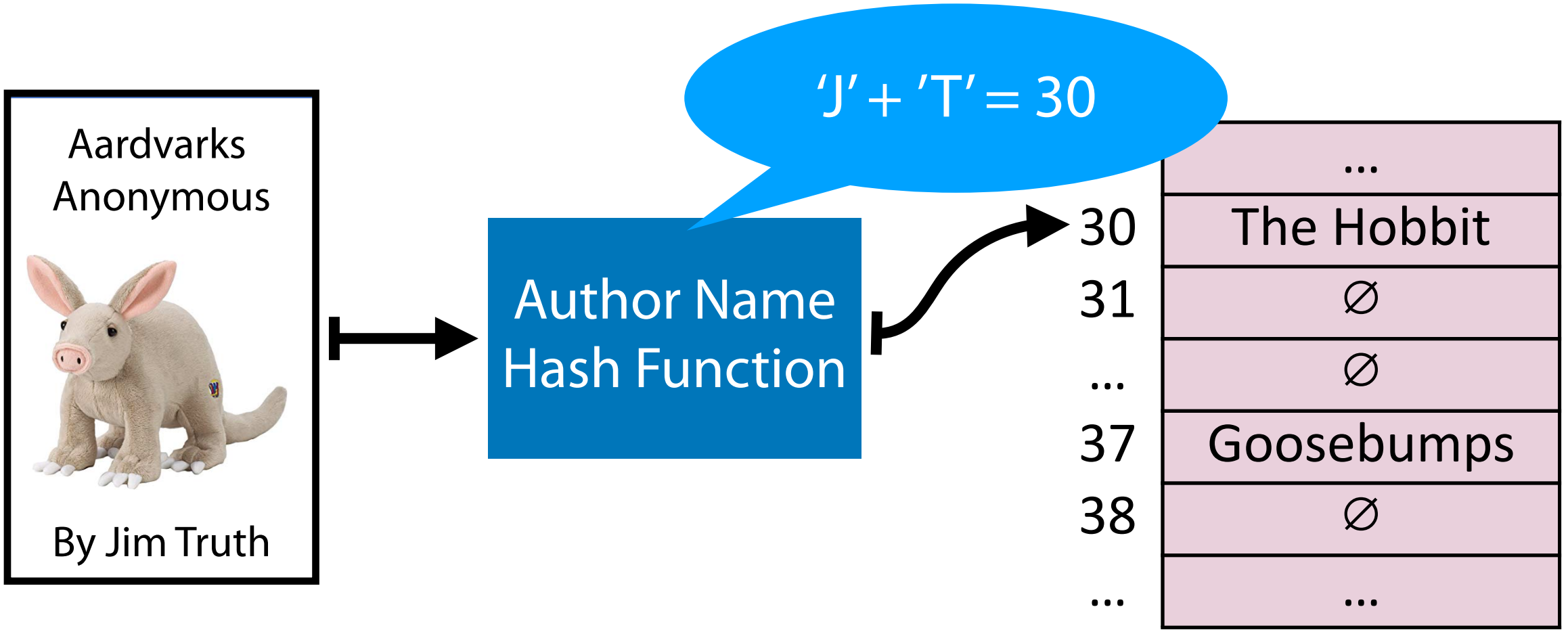


'R' + 'S' = 37

Author Name Hash Function

…	
30	The Hobbit
31	∅
…	∅
37	Goosebumps
38	∅
…	…

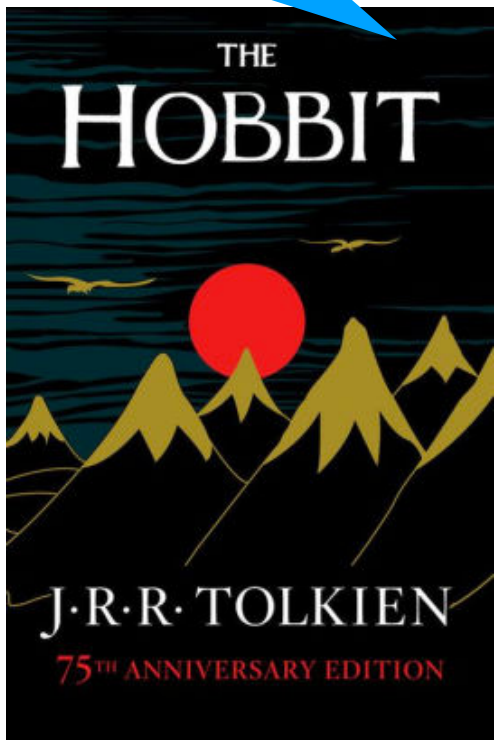
A Hash Table based Dictionary



Hash Collision

A ***hash collision*** occurs when multiple unique keys hash to the same value

J.R.R Tolkien = 30!



Jim Truth = 30!

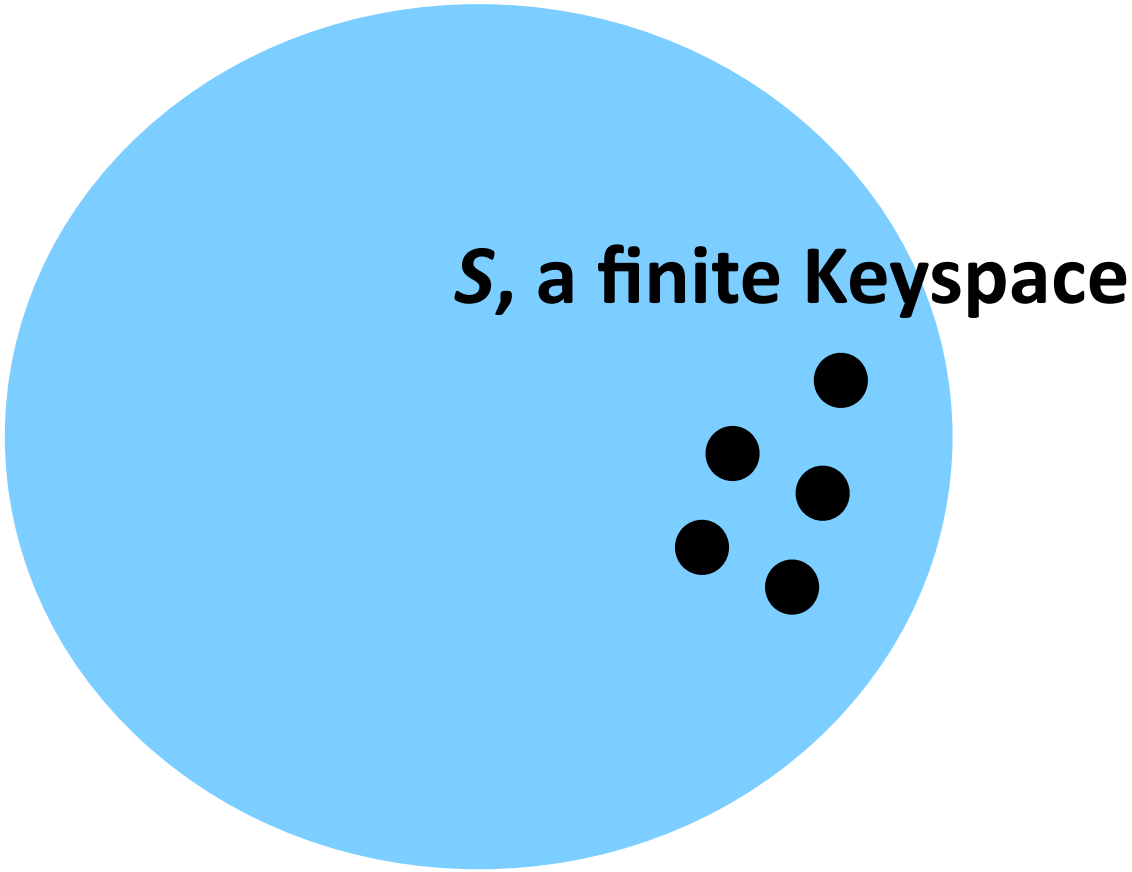
Aardvarks Anonymous

By Jim Truth

…	…
30	???
31	∅
…	∅
37	Goosebumps
38	∅
…	…

Perfect Hashing

If $m \geq S$, we can write a *perfect* hash with no collisions

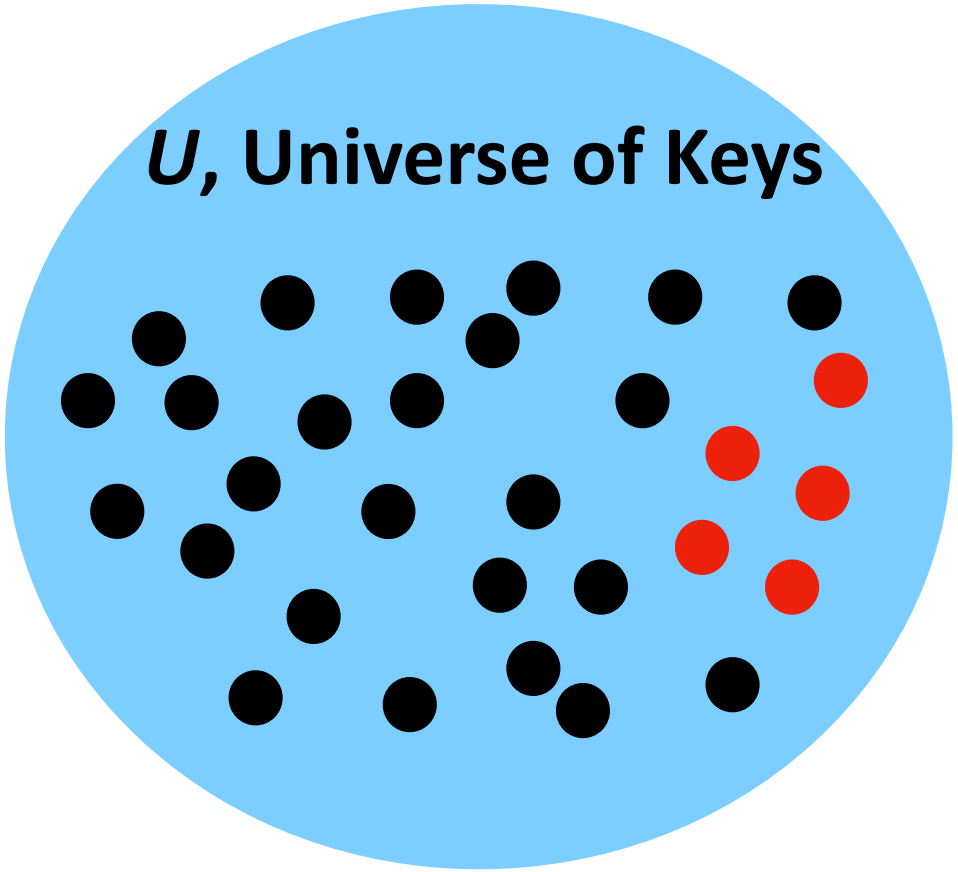


m elements

Key	Value

General Purpose Hashing

In CS 277, we want our hash functions to work *in general*.

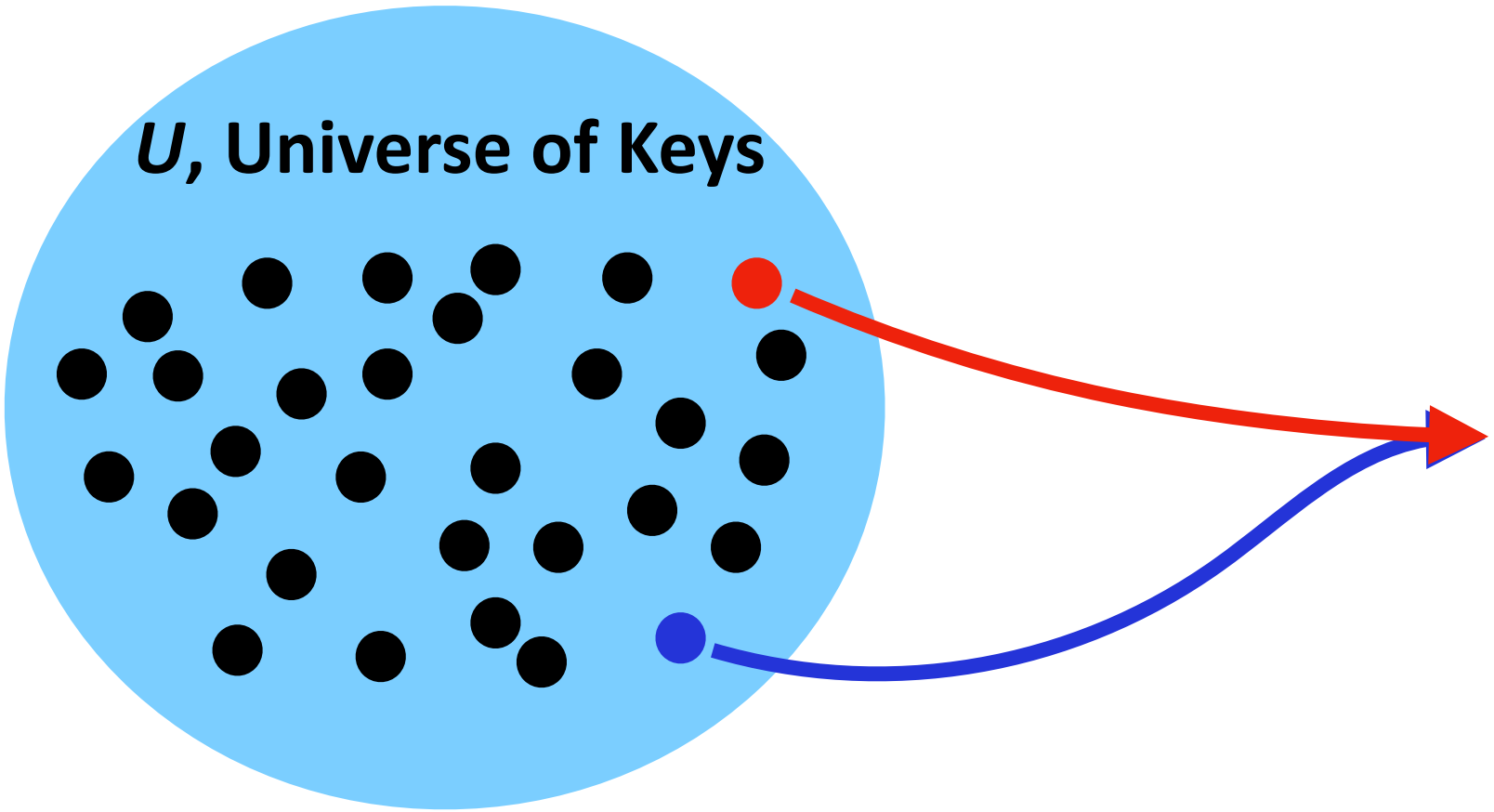


***m* elements**

Key	Value

General Purpose Hashing

If $m < U$, there must be at least one hash collision.



m elements

Key	Value

General Purpose Hashing

By fixing h, we open ourselves up to adversarial attacks.

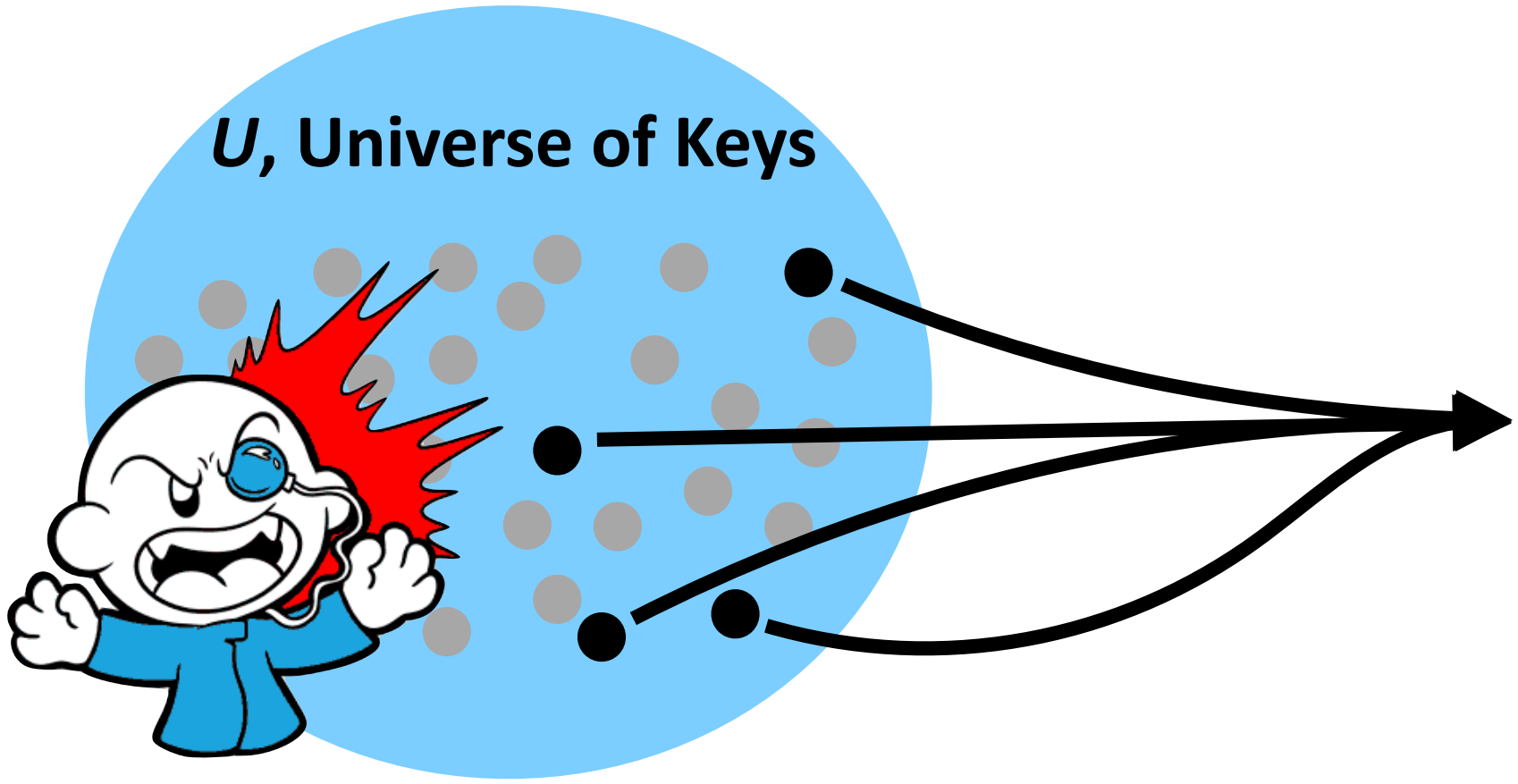


Image by Matthew Loffhagen

***m* elements**

Key	Value

A Hash Table based Dictionary

```
1 d = {}
2 d[k] = v
3 print(d[k])
```

A **Hash Table** consists of three things:

1. A hash function

2. A data storage structure

3. A method of addressing *hash collisions*

Open vs Closed Hashing

A hash function can be assessed by its likelihood of collisions.

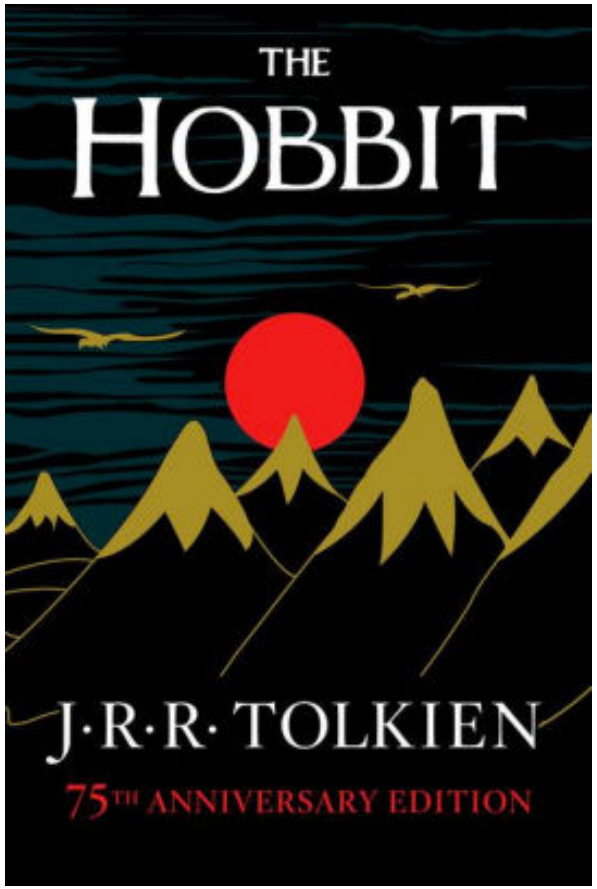
Addressing hash collisions depends on your storage structure.

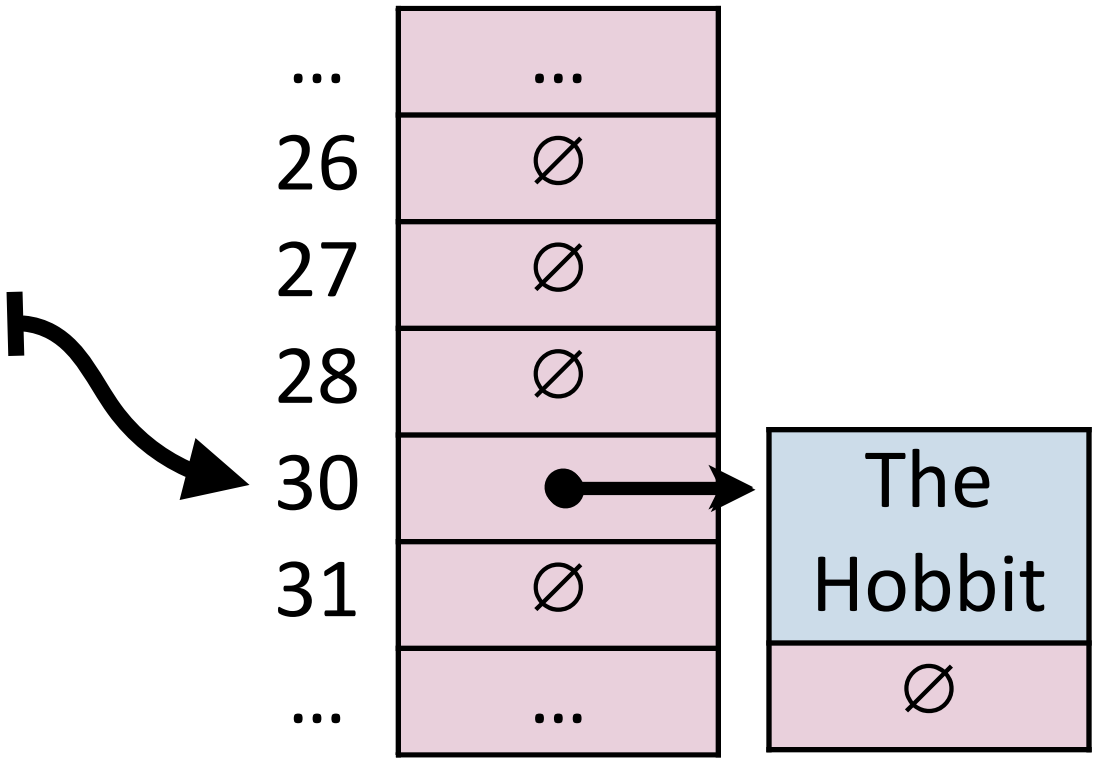
- **Open Hashing:**

- **Closed Hashing:**

Open Hashing

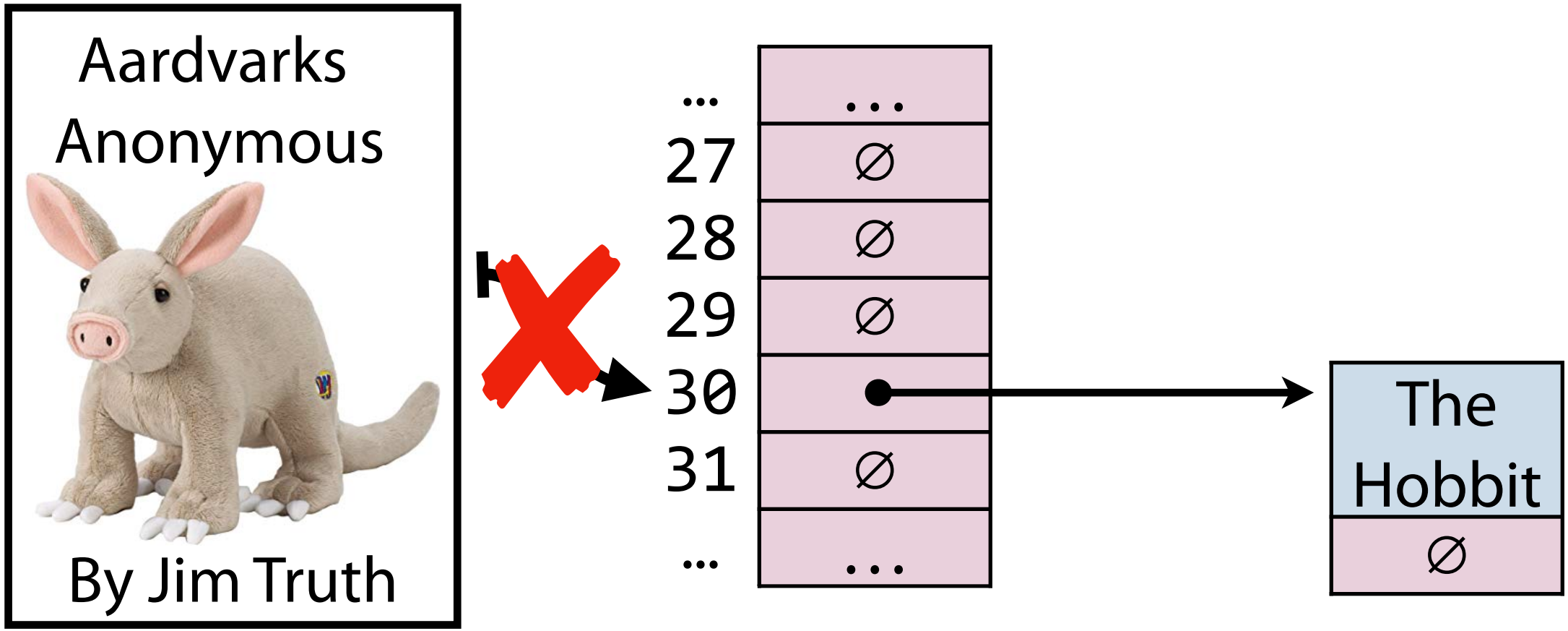
In an ***open hashing*** scheme, key-value pairs are stored externally (for example as a linked list).





Hash Collisions (Open Hashing)

A ***hash collision*** in an open hashing scheme can be resolved by _____. This is called ***separate chaining*.**



Insertion (Separate Chaining)

```
_insert("Bob")

_insert("Anna")
```

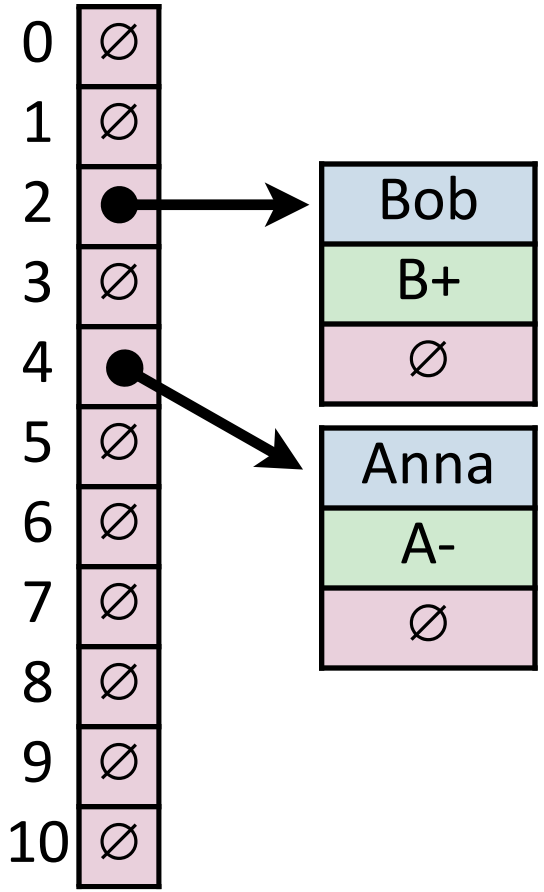
Key	Value	Hash
Bob	**B+**	**2**
Anna	**A-**	**4**
Alice	A+	4
Betty	B	2
Brett	A-	2
Greg	A	0
Sue	B	7
Ali	B+	4
Laura	A	7
Lily	B+	7

0	∅
1	∅
2	∅
3	∅
4	∅
5	∅
6	∅
7	∅
8	∅
9	∅
10	∅

Insertion (Separate Chaining)

```
_insert("Alice")
```

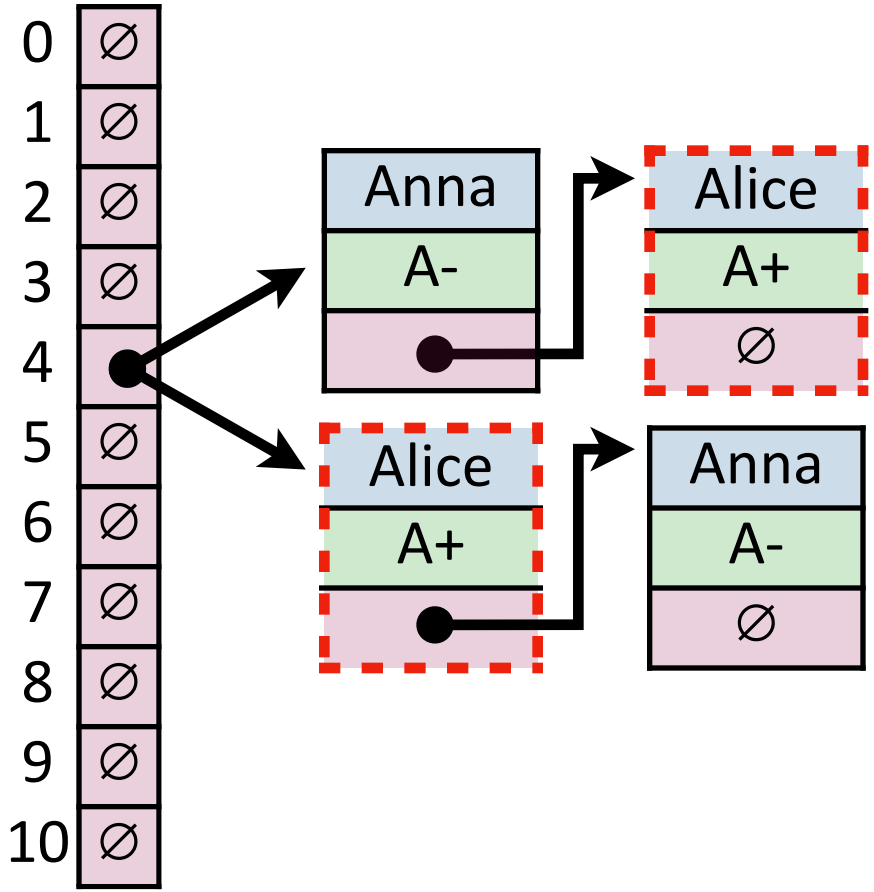
Key	Value	Hash
Bob	B+	2
Anna	A-	4
Alice	**A+**	**4**
Betty	B	2
Brett	A-	2
Greg	A	0
Sue	B	7
Ali	B+	4
Laura	A	7
Lily	B+	7



Insertion (Separate Chaining)

Where does Alice end up relative to Anna in the chain?

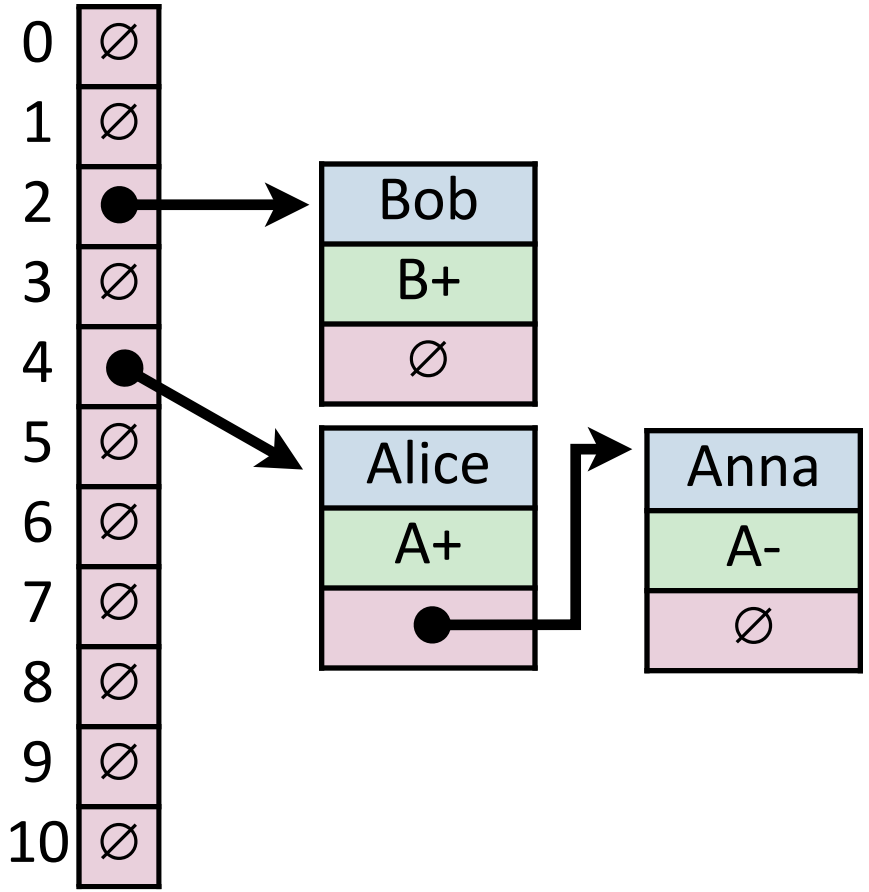
Key	Value	Hash
Bob	B+	2
Anna	A-	4
Alice	**A+**	**4**
Betty	B	2
Brett	A-	2
Greg	A	0
Sue	B	7
Ali	B+	4
Laura	A	7
Lily	B+	7



Join Code: 277

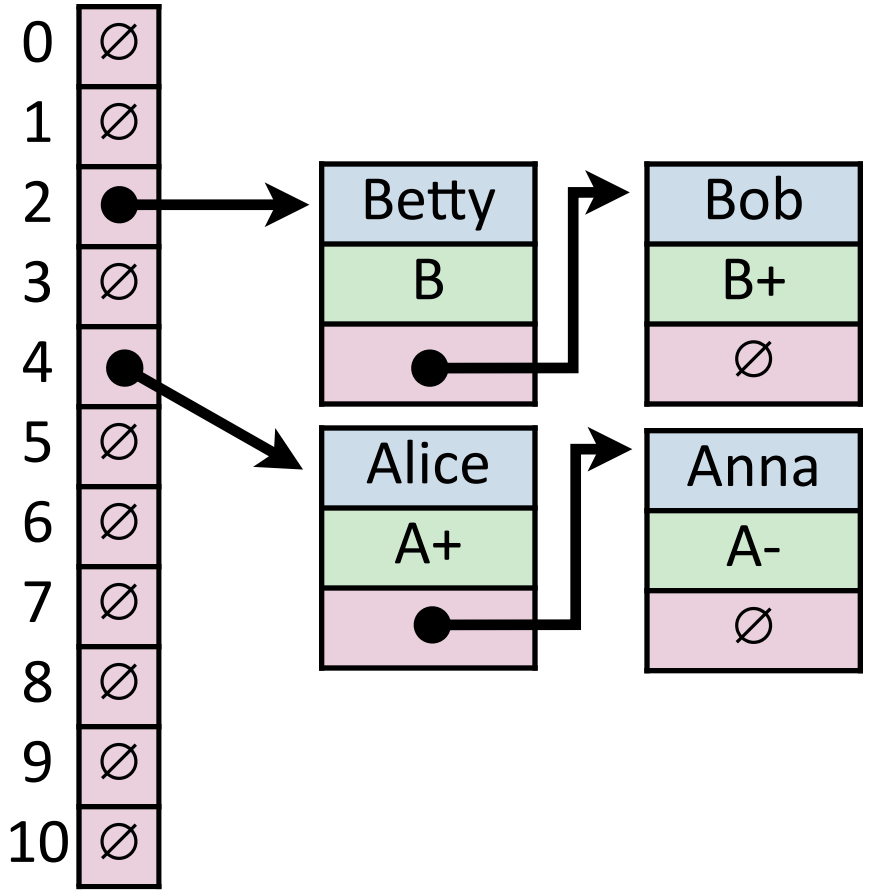
Insertion (Separate Chaining)

Key	Value	Hash
Bob	B+	2
Anna	A-	4
Alice	A+	4
Betty	**B**	**2**
Brett	A-	2
Greg	A	0
Sue	B	7
Ali	B+	4
Laura	A	7
Lily	B+	7



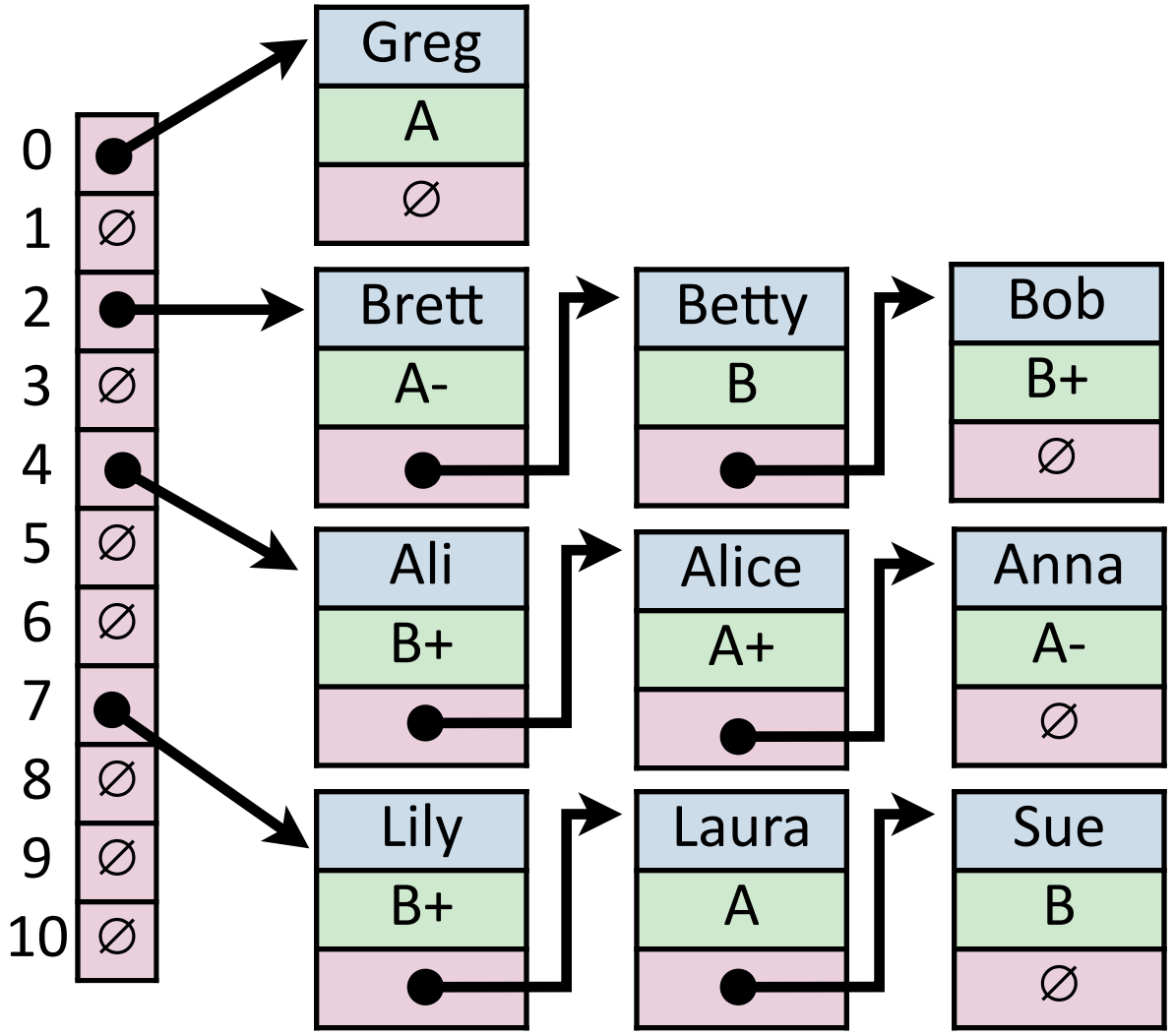
Insertion (Separate Chaining)

Key	Value	Hash
Bob	B+	2
Anna	A-	4
Alice	A+	4
Betty	**B**	**2**
Brett	A-	2
Greg	A	0
Sue	B	7
Ali	B+	4
Laura	A	7
Lily	B+	7



Insertion (Separate Chaining)

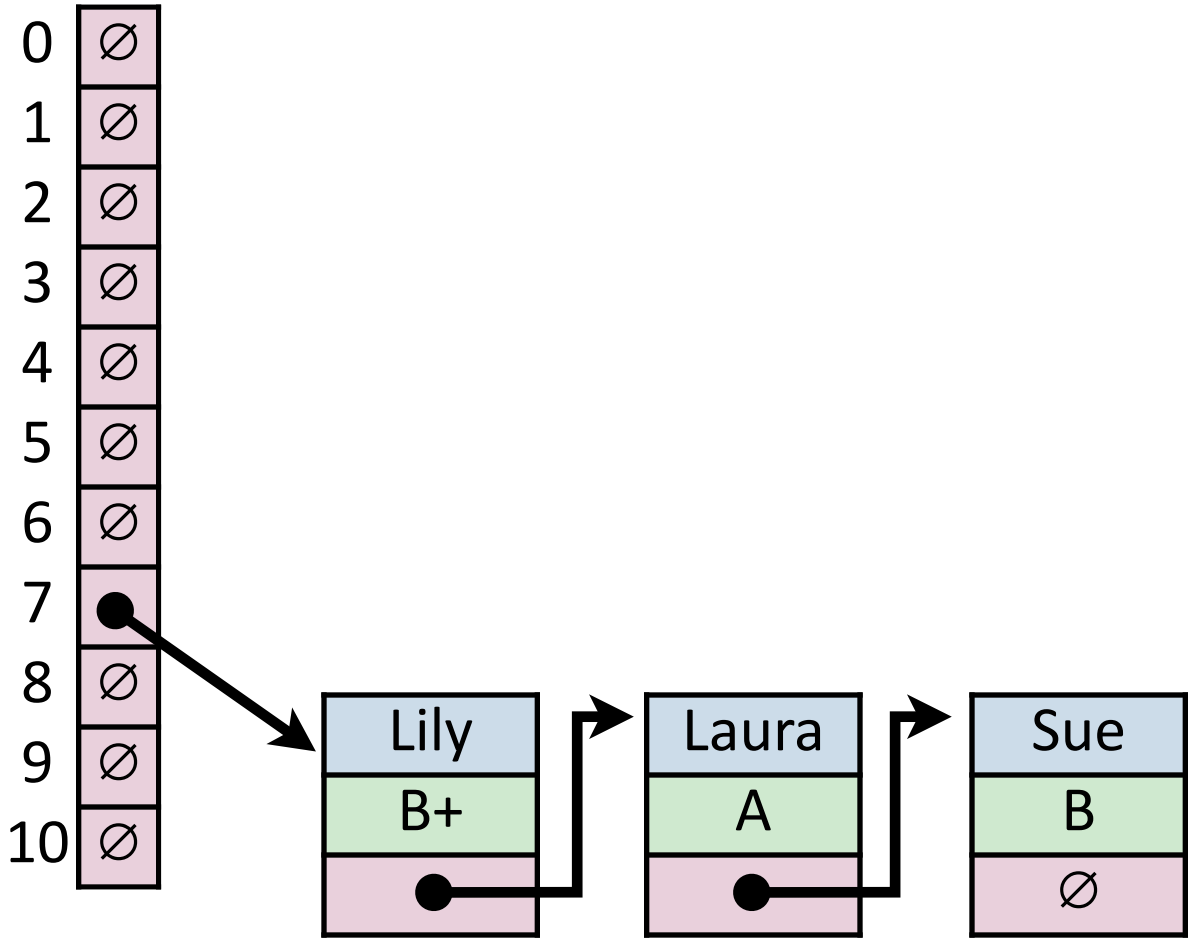
Key	Value	Hash
Bob	B+	2
Anna	A-	4
Alice	A+	4
Betty	B	2
Brett	A-	2
Greg	A	0
Sue	B	7
Ali	B+	4
Laura	A	7
Lily	B+	7



Find (Separate Chaining)

```
_find("Sue")
```

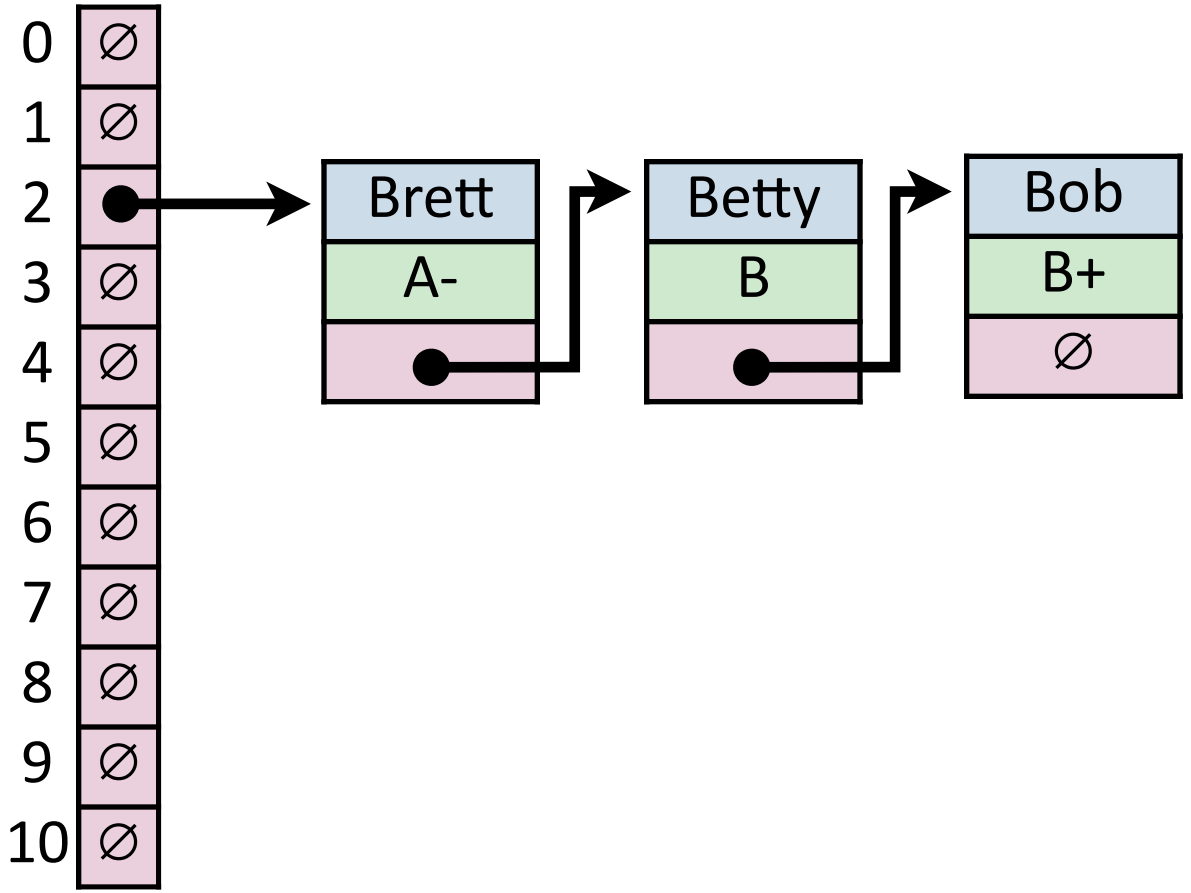
Key	Hash
Sue	7



Remove (Separate Chaining)

`_remove("Betty")`

Key	Hash
Betty	2



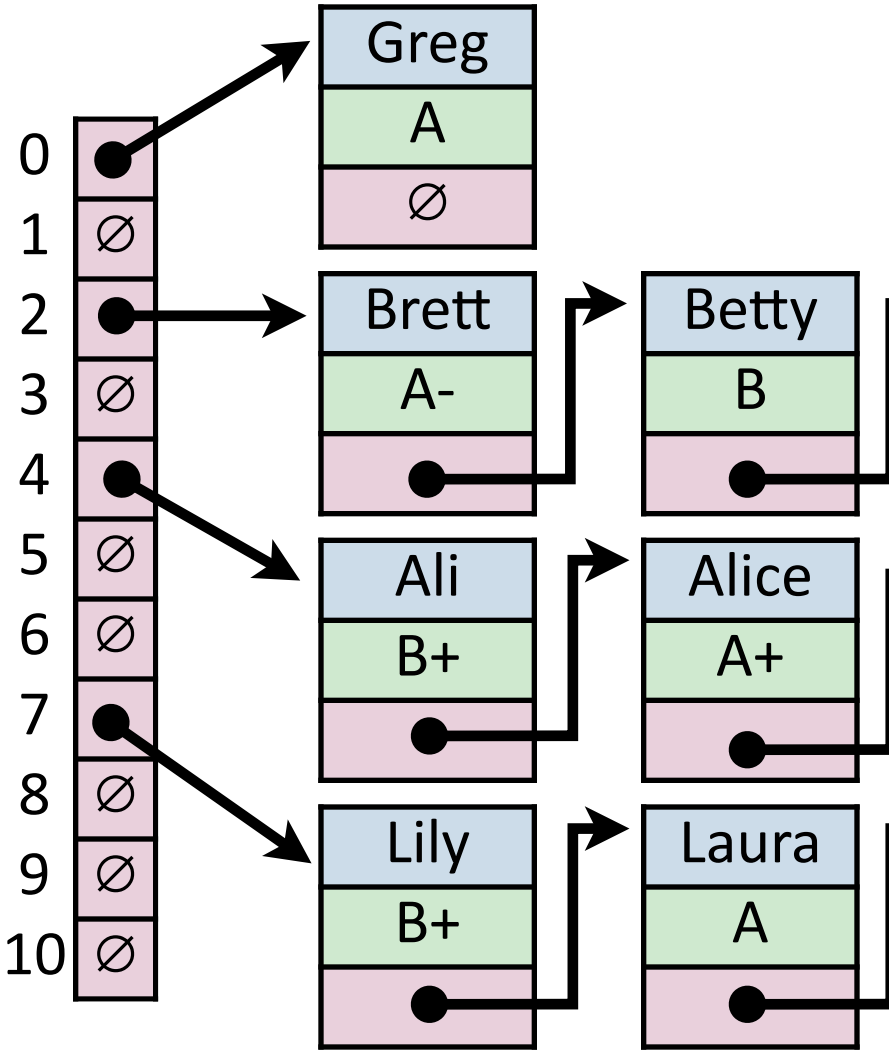
Hash Table (Separate Chaining)

For hash table of size *m* and *n* elements:

Find runs in: _____

Insert runs in: _____

Remove runs in: _____



Fundamentals of Probability

Imagine you roll a pair of six-sided dice.

The **sample space** Ω is the set of all possible outcomes.

An **event** $E \subseteq \Omega$ is any subset.

Fundamentals of Probability

Imagine you roll a pair of six-sided dice. What is the expected value?

The **expectation** of a (discrete) random variable is:

$$E[X] = \sum_{x \in \Omega} Pr\{X = x\} \cdot x$$

Fundamentals of Probability

Imagine you roll a pair of six-sided dice. What is the expected value?

Linearity of Expectation: For any two random variables X and Y,

$$E[X + Y] = E[X] + E[Y]$$

Fundamentals of Probability

Imagine you roll a pair of six-sided dice. What is the expected value?

Linearity of Expectation: For any two random variables X and Y,

$$E[X + Y] = E[X] + E[Y]$$

$$= \sum_x \sum_y Pr\{X = x, Y = y\}(x + y)$$

Fundamentals of Probability

Imagine you roll a pair of six-sided dice. What is the expected value?

Linearity of Expectation: For any two random variables X and Y,

$$E[X+Y] = E[X] + E[Y]$$

$$= \sum_x \sum_y Pr\{X = x, Y = y\}(x+y)$$

$$= \sum_x x \sum_y Pr\{X = x, Y = y\} + \sum_y y \sum_x Pr\{X = x, Y = y\}$$

Fundamentals of Probability

Imagine you roll a pair of six-sided dice. What is the expected value?

Linearity of Expectation: For any two random variables X and Y,

$$E[X+Y] = E[X] + E[Y]$$

$$= \sum_x \sum_y Pr\{X = x, Y = y\}(x + y)$$

$$= \sum_x x \sum_y Pr\{X = x, Y = y\} + \sum_y y \sum_x Pr\{X = x, Y = y\}$$

$$= \sum_x x \cdot Pr\{X = x\} + \sum_y y \cdot Pr\{Y = y\}$$

Fundamentals of Probability

Imagine you roll a pair of six-sided dice. What is the expected value?

Linearity of Expectation: For any two random variables X and Y,

$$E[X + Y] = E[X] + E[Y]$$

Hash Table

Worst-Case behavior is bad — but what about randomness?

1) **Fix *h***, our hash, and assume it is good **for *all keys***:

2) Create a ***universal hash function family:***

Simple Uniform Hashing Assumption

Given table of size m, a simple uniform hash, h, implies

$$\forall k_1, k_2 \in U \text{ where } k_1 \neq k_2 \text{ , } Pr(h[k_1] = h[k_2]) = \frac{1}{m}$$

Uniform:

Independent:

Separate Chaining Under SUHA

Table Size: m

Num objects: n

Claim: Under SUHA, expected length of chain is $\frac{n}{m}$

$\alpha_j =$ expected # of items hashing to position j

$$\alpha_j = \sum_i H_{i,j}$$

$$H_{i,j} = \begin{cases} 1 \text{ if item i hashes to j} \\ 0 \text{ otherwise} \end{cases}$$

Separate Chaining Under SUHA

Table Size: m

Num objects: n

Claim: Under SUHA, expected length of chain is $\frac{n}{m}$

$\alpha_j =$ expected # of items hashing to position j

$$\alpha_j = \sum_i H_{i,j}$$

$$H_{i,j} = \begin{cases} 1 \text{ if item i hashes to j} \\ 0 \text{ otherwise} \end{cases}$$

$$E[\alpha_j] = E\left[\sum_i H_{i,j}\right]$$

Separate Chaining Under SUHA

Table Size: m

Num objects: n

Claim: Under SUHA, expected length of chain is $\frac{n}{m}$

$\alpha_j =$ expected # of items hashing to position j

$$\alpha_j = \sum_i H_{i,j}$$

$$H_{i,j} = \begin{cases} 1 \text{ if item i hashes to j} \\ 0 \text{ otherwise} \end{cases}$$

$$E[\alpha_j] = E[\sum_i H_{i,j}]$$

$$E[\alpha_j] = \sum_i Pr(H_{i,j} = 1) * 1 + Pr(H_{i,j} = 0) * 0$$

Separate Chaining Under SUHA

Table Size: m

Num objects: n

Claim: Under SUHA, expected length of chain is $\frac{n}{m}$

$\alpha_j =$ expected # of items hashing to position j

$$\alpha_j = \sum_i H_{i,j}$$

$$H_{i,j} = \begin{cases} 1 \text{ if item i hashes to j} \\ 0 \text{ otherwise} \end{cases}$$

$$E[\alpha_j] = E[\sum_i H_{i,j}]$$

$$E[\alpha_j] = \sum_i Pr(H_{i,j} = 1) * 1 + Pr(H_{i,j} = 0) * 0$$

$$E[\alpha_j] = n * Pr(H_{i,j} = 1)$$

Separate Chaining Under SUHA

Table Size: m

Num objects: n

Claim: Under SUHA, expected length of chain is $\frac{n}{m}$

$\alpha_j =$ expected # of items hashing to position j

$$\alpha_j = \sum_i H_{i,j}$$

$$H_{i,j} = \begin{cases} 1 \text{ if item i hashes to j} \\ 0 \text{ otherwise} \end{cases}$$

$$E[\alpha_j] = E[\sum_i H_{i,j}]$$

$$Pr[H_{i,j} = 1] = \frac{1}{m}$$

$$E[\alpha_j] = n * Pr(H_{i,j} = 1)$$

Separate Chaining Under SUHA

Claim: Under SUHA, expected length of chain is $\frac{n}{m}$

Table Size: m

Num objects: n

α_j = expected # of items hashing to position j

$$\alpha_j = \sum_i H_{i,j}$$

$$H_{i,j} = \begin{cases} 1 \text{ if item i hashes to j} \\ 0 \text{ otherwise} \end{cases}$$

$$E[\alpha_j] = E[\sum_i H_{i,j}]$$

$$Pr[H_{i,j} = 1] = \frac{1}{m}$$

$$E[\alpha_j] = n * Pr(H_{i,j} = 1)$$

$$\mathbf{E[\alpha_j] = \frac{n}{m}}$$

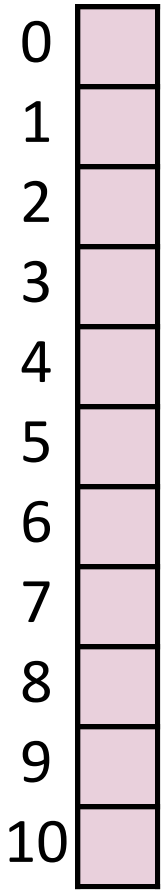
Separate Chaining Under SUHA

Under SUHA, a hash table of size *m* and *n* elements:

Find runs in: _____.

Insert runs in: _____.

Remove runs in: _____.



Separate Chaining Under SUHA

Pros:

Cons:

Next time: Closed Hashing

Closed Hashing: store *k,v* pairs in the hash table

S = { 1, 8 , 15}

h(k) = k % 7